Document de spécification

Générée le 29/04/2026 à 09:39

Modèle PRAPS

Pastoralisme au Sahel

Version : 1.0.0

Date de version : 29/04/2026

Auteurs

Tingouro Sanogo

Affiliation : Doctorant

Rôle : Auteur principal

Email : `tingourosanogo@gmail.com`

Pr. Jean-Pierre Müller

Affiliation : Chercheur senior - CIRAD

Rôle : Auteur principal

Email : `jean-pierre.muller@cirad.fr`

Dr Oumar Y. Maïga

Affiliation : Enseignant-chercheur

Rôle : Auteur principal

Email : `maigabababa78@yahoo.fr`

Dr Souleymane S. Traoré

Affiliation : Enseignant-chercheur

Rôle : Auteur principal

Email : `sstraore@yahoo.fr`

Métadonnées

Mots-clés : `Modèle à base d'agents`, `Pastoralisme`, `Sahel`

Langue : FR

Document généré automatiquement le 29/04/2026 à 09:39

Table des Matières

1. Introduction
2. Structure et dynamiques
3. Initialisation
4. Indicateurs
5. Mode d'Emploi

Introduction

Principe et définitions

Toute modélisation part d'une question de la forme :

Quel est l'impact de X sur Y étant donné Z ?

dans laquelle :

- X définit l'ensemble des scénarios que l'on souhaite explorer
- Y l'ensemble des indicateurs que l'on souhaite observer comme réponse aux différents scénarios
- Z le contexte de la question

Concepts fondamentaux

- **Système** : Système structuré en entités et processus

Types d'attributs :

- **parametres** : dont la valeur ne varie pas au fil de la simulation
- **variables_etat** : dont la simulation fait varier la valeur au fil du temps

Portée des attributs :

- globaux : partagés par tout le système (nous avons choisi de les décrire dans l'entité « système »)
- locaux : partagés par entités de même type
- individuels : spécifiques à chaque entité

Relations questions-modèle

Formalisation :

```
X ⊆ A, Z ⊆ A, X ∩ Z = ∅, X ∪ Z = A
```

```
X est un sous-ensemble des attributs (de A)
```

```
Z est un sous-ensemble des attributs disjoint de X
```

```
X union Z = A
```

```
Y est calculé à partir des attributs
```

Implication : Un choix différent de X et Z permet de répondre à d'autres questions avec le même modèle On peut donc imaginer l'ensemble des questions auxquelles un même modèle peut répondre.

Le modèle

La question retenue pour le modèle est :

Quel est l'impact des schémas de gestion et des facteurs exogènes sur la durabilité des ressources naturelles ?

Domaine : socio-écosystème agricole

Échelle : territoriale

Représentations graphiques

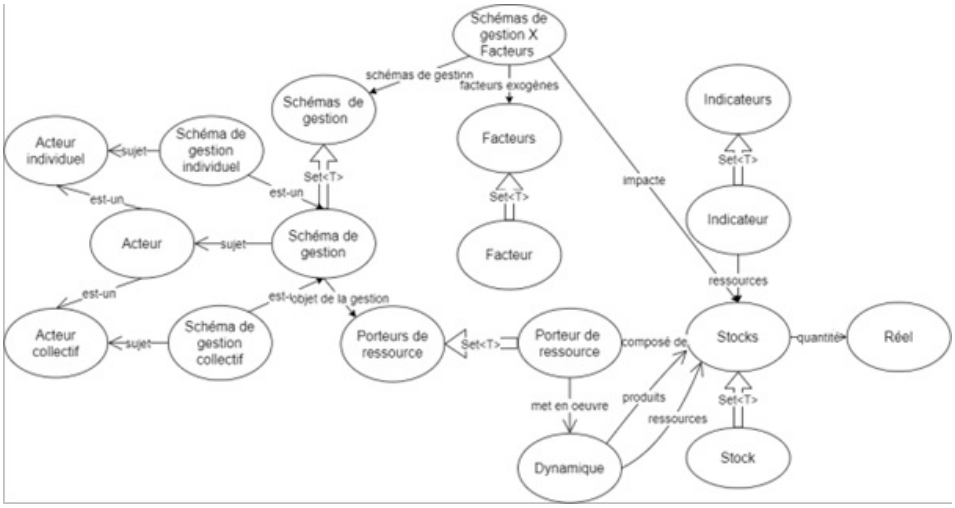


Figure 1 : Question de recherche schématisée

Sur la flèche diagonale, on représente qu'une combinaison (X) de schémas de gestion et de facteurs impacte les stocks. En bas à droite, les stocks sont des quantités de ressource pour différents usages (par les humains, les animaux, les plantes et autres processus biophysiques). Au milieu en bas, un porteur de ressource est composé de divers stocks et met en œuvre une dynamique qui utilise des stocks comme ressources et produits des stocks. A gauche, les schémas de gestion sont des ensembles de schémas de gestion, mis-en-oeuvre par un acteur (sujet). On distingue les schémas de gestion individuels des schémas de gestion collectif mis-en-oeuvre par les auteurs correspondant. Dans notre cas, les acteurs individuels sont des exploitations; les acteurs collectifs sont des villages. Au milieu en haut, les facteurs exogènes sont un ensemble de facteurs, un facteur pouvant être comme suit

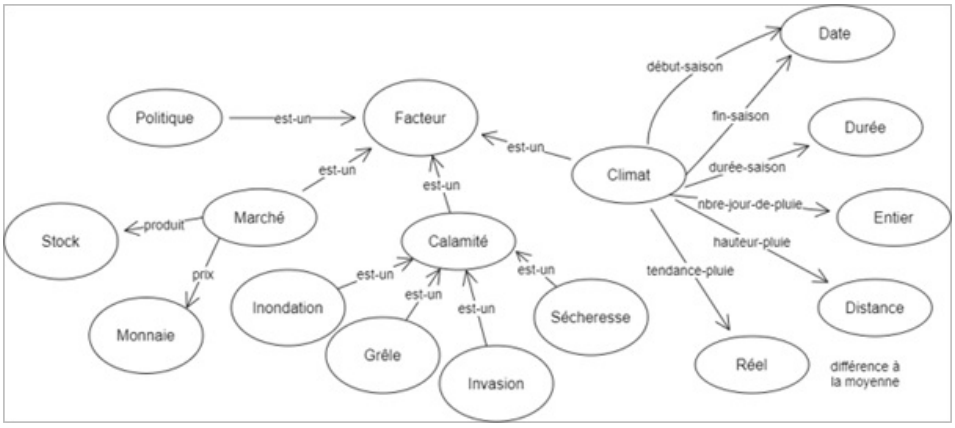


Figure 2 : Facteurs exogènes

Les facteurs exogènes sont un ensemble de facteurs influençant le système sans être contrôlés par les acteurs.

Composants principaux

Porteurs_Ressources

Description : Composés de divers stocks et mettent en œuvre des dynamiques

Rôle : Utilisent des stocks comme ressources et produisent des stocks

Schemas_Gestion

Description : Ensembles de schémas de gestion mis en œuvre par des acteurs

Types :

- individuels : par exploitation
- collectifs : par village

Stocks

Description : Quantités de ressources pour différents usages

Usages :

- humains
- animaux
- plantes
- processus biophysiques

Acteurs du système

- **Individuels** : exploitations agricoles
- **Collectifs** : villages, coopératives

Contextualisation

Domaine d'application : agroécologie, gestion des ressources naturelles

Pertinence scientifique : Modélisation des interactions complexes entre pratiques de gestion et dynamiques des ressources dans un contexte de changement global

Public cible : - décideurs politiques - chercheurs en socio-écosystèmes - acteurs territoriaux

Structure et dynamiques

Ecosystem

- **Name**: Migrated Livestock Module
- **Location**: Agricultural Zone

Entité : Soil (agent_001)

Documentation

Soil

Attributs

Les ressources détenues par l'agent :

Paramètre	Commentaire	Unité
`soil_nitrogen`	Nitrogen in the soil, essential for fertility	kg
`soil_carbon`	Carbon in the soil, linked to organic matter	kg
`leaf_litter`	Leaf litter	None
`natural_litter`	Natural litter	None
`cultivated_litter`	Cultivated litter	None
`fertilizer`	Fertilizer	None
`soil_type`	Type of soil	-
`manure`	Manure	kg
`field_capacity`	The maximum volumetric water content (fraction) the soil can hold against gravity.	proportion

Dynamiques

Processus: mineralization

Documentation :

soil mineralization, converting organic matter (including manure) to enrich nitrogen/carbon

Attributs

Le processus dépend des variables suivantes :

Variables d'état	Entité	Définition	Unité
leaf_litter	Soil	Leaf litter	None
natural_litter	Soil	Natural litter	None
cultivated_litter	Soil	Cultivated litter	None
manure	Soil	Manure	kg
soil_moisture	Soil	Soil Moisture	%
soil_temperature	Soil	Soil Temperature	°C
soil_pH	Soil	Soil pH	pH
field_capacity	Soil	The maximum volumetric water content (fraction) the soil can hold against gravity.	proportion

Le processus agit sur les variables suivantes :

Variables d'état	Entité	Définition	Unité
soil_nitrogen	Soil	Nitrogen in the soil, essential for fertility	kg
soil_carbon	Soil	Carbon in the soil, linked to organic matter	kg
leaf_litter	Soil	Leaf litter	None
natural_litter	Soil	Natural litter	None
cultivated_litter	Soil	Cultivated litter	None
manure	Soil	Manure	kg

et dépend des paramètres suivants :

Paramètres	Entité	Définition	Unité
base_bineralization_rate_k_base	Parameter	The base rate constant of organic matter decomposition and mineralization under optimal conditions (T=10°C, Moisture=Field Capacity, pH=7).	proportion/step
natural_litter_to_c	Parameter	Natural Litter to Carbon	kgC/kgDM
natural_litter_to_n	Parameter	Natural Litter to Nitrogen	kgN/kgDM
cultivated_litter_to_c	Parameter	Cultivated Litter to Carbon	kgC/kgDM
cultivated_litter_to_n	Parameter	The proportional mass of Nitrogen (N) content in a unit of Cultivated Litter.	kgN/kgDM
leaf_litter_to_n	Parameter	Leaf Litter to Nitrogen	kgN/kgDM
leaf_litter_to_c	Parameter	Leaf Litter to Carbon	kgC/kgDM
manure_to_n	Parameter	The proportional mass of Nitrogen (N) content in a unit of Manure.	kgN/kgDM
manure_to_c	Parameter	The proportional mass of Carbon (C) content in a unit of Manure.	kgC/kgDM
cn_ratio	Parameter	The Carbon-to-Nitrogen ratio of the bulk soil organic matter, influencing mineralization and immobilization.	dimensionless

Comportement

Description détaillée du processus (équations, états, activités, etc.).

Équation :

`n_min = k_base * f(T) * f(θ) * f(pH) * (c/n) * SOM` `n_input_total = sum(n_content * Stock)`

Pseudo-code

Pseudo-code tel que fourni par le modèle.

```
 // Step 1: calculate Environmental Rate Factor (k) based on temp, moisture, pH, c/n ratio
// Temperature effect
soil_temp = soil_temperature /* resource_063 */;
temp_factor = exp(0.069 * (soil_temp - 10));
if (temp_factor < 0) {
  temp_factor = 0;
}

// Moisture effect
soil_moisture = soil_moisture /* resource_053 */;
field_capacity_tmp = field_capacity /* resource_005 */;
moisture_factor = (soil_moisture / field_capacity_tmp);
if (moisture_factor > 1) {
  moisture_factor = 1;
}
if (moisture_factor < 0.2) {
  moisture_factor = 0;
}

// pH effect
soil_pH_tmp = soil_pH /* resource_052 */;
ph_factor = min(1, max(0, (soil_pH_tmp - 4)/(7 - 4)));

// c/n ratio effect
cn_ratio_tmp = cn_ratio /* property_032 */;
cn_factor = min(1, max(0, (25 - cn_ratio_tmp)/15));

// calculate mineralization rate
k_base = base_mineralization_rate_k_base /* property_033 */;
k = k_base * temp_factor * moisture_factor * ph_factor * cn_factor;

// Step 2: Determine actual mass to mineralize (Robustness check)
rate = k * time_step; // k is the proportion of stock mineralized

// consumable organic stocks (current values)
litter_leaf_stock = leaf_litter /* resource_006 */;
litter_natural_stock = litter_natural /* resource_007 */;
litter_cultivated_stock = litter_cultivated /* resource_008 */;
manure_stock = manure /* resource_031 */;

// calculate consumed mass for each stock, capped at current stock availability
consumed_leaf = min(litter_leaf_stock * rate, litter_leaf_stock);
consumed_natural = min(litter_natural_stock * rate, litter_natural_stock);
consumed_cultivated = min(litter_cultivated_stock * rate, litter_cultivated_stock);
consumed_manure = min(manure_stock * rate, manure_stock);

// Step 3: calculate Mineralized n and c
n_from_leaf = consumed_leaf * leaf_litter_to_n /* property_046 */;
n_from_natural = consumed_natural * natural_litter_to_n /* property_002 */;
n_from_cultivated = consumed_cultivated * cultivated_litter_to_n /* property_083 */;
n_from_manure = consumed_manure * manure_to_n /* property_085 */;
n_mineralized = n_from_leaf + n_from_natural + n_from_cultivated + n_from_manure; // Total Mineralize

c_from_leaf = consumed_leaf * leaf_litter_to_c /* property_004 */;
c_from_natural = consumed_natural * natural_litter_to_c /* property_001 */;
c_from_cultivated = consumed_cultivated * cultivated_litter_to_c /* property_003 */;
c_from_manure = consumed_manure * manure_to_c /* property_084 */;
c_mineralized = c_from_leaf + c_from_natural + c_from_cultivated + c_from_manure; // Total Mineralize
```

```
// Step 4: Update Resources (Mineral Pools increase, Organic Stocks decrease)
soil_nitrogen /* resource_001 */ = soil_nitrogen + n_mineralized;
soil_carbon /* resource_002 */ = soil_carbon + c_mineralized;

leaf_litter /* resource_006 */ = leaf_litter - consumed_leaf;
natural_litter /* resource_007 */ = natural_litter - consumed_natural;
cultivated_litter /* resource_008 */ = cultivated_litter - consumed_cultivated;
manure /* resource_031 */ = manure - consumed_manure;
```

Entité : TreeCover (agent_002)

Documentation

Tree cover

Attributs

Les ressources détenues par l'agent :

Paramètre	Commentaire	Unité
`tree_type`	Type of tree cover	None
`tree_count`	Number of trees	None
`biomass`	Total standing above-ground mass of the Tree Cover.	kg

Dynamiques

Processus: tree_cover_growth

Documentation :

Growth of perennial Tree Biomass, limited by rainfall and soil nitrogen.

Attributs

Le processus dépend des variables suivantes :

Variables d'état	Entité	Définition	Unité
biomass	TreeCover	Total standing above-ground mass of the Tree Cover.	kg
rainfall	ClimateSystem	Rainfall	mm/year
soil_nitrogen	Soil	Nitrogen in the soil, essential for fertility	kg

Le processus agit sur les variables suivantes :

Variables d'état	Entité	Définition	Unité

Variables d'état	Entité	Définition	Unité
biomass	TreeCover	Total standing above-ground mass of the Tree Cover.	kg
soil_nitrogen	Soil	Nitrogen in the soil, essential for fertility	kg

et dépend des paramètres suivants :

Paramètres	Entité	Définition	Unité
residue_grazing_prop	Parameter	Proportion of crop residue consumed by animals after harvest.	proportion
tree_n_uptake_ratio	Parameter	Proportional mass of Nitrogen required per unit of tree biomass growth.	kgN/kgDM
rainfall_normal	Parameter	Rainfall Normal	mm
n_opt	Parameter	Optimal soil nitrogen for plant growth	kgN/ha

Comportement

Description détaillée du processus (équations, états, activités, etc.).

Équation :

```
TreeGrowth = r_max * Biomass * f(Rain) * f(N)
```

Pseudo-code

Pseudo-code tel que fourni par le modèle.

```
 // 1. Get Environmental Limiting Factors
r_max = tree_growth_rate /* property_124 */;
current_biomass = biomass /* resource_027 */;

// Use existing environmental parameters
rainfall_tmp = rainfall /* resource_010 */;
optimal_rainfall_tmp = optimal_rainfall /* property_031 */;
rain_factor = min(1, rainfall_tmp / optimal_rainfall_tmp);

soil_nitrogen_current = soil_nitrogen /* resource_001 */;
n_opt_tmp = n_opt /* property_097 */;
n_factor = min(1, soil_nitrogen_current / n_opt_tmp);

// 2. Calculate Potential and Absolute Growth
growth_rate = r_max * rain_factor * n_factor;

tree_growth = current_biomass * growth_rate;

// 3. Update Biomass Resource
biomass /* resource_027 */ = biomass + tree_growth;

// 4. Nitrogen Uptake (Mass Balance with Robustness Check)
n_uptake_ratio = tree_n_uptake_ratio /* property_055 */;

// Calculate needed uptake
n_consumption_needed = tree_growth * n_uptake_ratio;

// Deduct the minimum of what is needed and what is currently available
n_deducted = min(n_consumption_needed, soil_nitrogen_current);
soil_nitrogen /* resource_001 */ = soil_nitrogen - n_deducted;
```

Processus: litter_fall

Documentation :

Litter fall and drought-induced mortality from perennial Tree Cover biomass to soil organic stocks.

Attributs

Le processus dépend des variables suivantes :

Variables d'état	Entité	Définition	Unité
biomass	TreeCover	Total standing above-ground mass of the Tree Cover.	kg
leaf_litter	Soil	Leaf litter	None
natural_litter	Soil	Natural litter	None
soil_moisture	Soil	Soil Moisture	%
field_capacity	Soil	The maximum volumetric water content (fraction) the soil can hold against gravity.	proportion

Le processus agit sur les variables suivantes :

Variables d'état	Entité	Définition	Unité
biomass	TreeCover	Total standing above-ground mass of the Tree Cover.	kg
leaf_litter	Soil	Leaf litter	None
natural_litter	Soil	Natural litter	None

et dépend des paramètres suivants :

Paramètres	Entité	Définition	Unité
litter_fall_rate	Parameter	The proportional rate (per time step) at which perennial biomass converts to litter fall.	proportion/step
drought_mortality_rate	Parameter	Drought Mortality Rate	proportion/step

Comportement

Description détaillée du processus (équations, états, activités, etc.).

Équation :

`LitterFall = (LitterRate + DroughtFactor) * Biomass` `DeltaBiomass = -LitterFall`

Pseudo-code

Pseudo-code tel que fourni par le modèle.

```
 // 1. Get Biomass and Rates
biomass_stock = Biomass /* resource_027 */;
base_litter_rate = litter_fall_rate /* property_054 */;
drought_mortality_rate = drought_mortality_rate /* property_036 */;

// 2. Get Environmental Condition for Stress Calculation
soil_moisture = soil_moisture /* resource_053 */;
field_capacity = field_capacity /* resource_005 */;

// Calculate Drought Stress Factor: Scales from 0 (low stress) to 1 (max stress)
// We use max(0, 1 - ...) to prevent negative stress values if moisture exceeds field capacity
drought_stress_factor = max(0, 1 - (soil_moisture / field_capacity));

// Calculate Total Fall Rate: Base rate + stress-induced rate
total_fall_rate = base_litter_rate + (drought_mortality_rate * drought_stress_factor);

// 3. Calculate actual litter fall amount (Robustness Check)
litter_fall_amount_potential = biomass_stock * total_fall_rate;

// The amount consumed cannot exceed the current stock
litter_fall_amount = min(litter_fall_amount_potential, biomass_stock);

// Split litter fall between two natural pools (e.g., 50/50 split)
litter_to_leaf = litter_fall_amount * 0.5;
litter_to_natural = litter_fall_amount * 0.5;

// 4. Update Resources (Mass Balance)
biomass /* resource_027 */ = biomass - litter_fall_amount;

leaf_litter /* resource_006 */ = leaf_litter + litter_to_leaf;
natural_litter /* resource_007 */ = natural_litter + litter_to_natural;
```

Entité : HerbaceousCover (agent_003)

Documentation

Herbaceous cover

Attributs

Les ressources détenues par l'agent :

Paramètre	Commentaire	Unité
`herbaceous_type`	Type of herbaceous cover	(Crop, natural prairie, cultivated prairie)
`seasonal_fodder`	Seasonal Fodder	None

Dynamiques

Processus: Herbaceous_cover_growth

Documentation :

Herbaceous cover growth, limited by environment (Rainfall, Soil N) and reduced by grazing pressure.

Attributs

Le processus dépend des variables suivantes :

Variables d'état	Entité	Définition	Unité
rainfall	ClimateSystem	Rainfall	mm/year
soil_nitrogen	Soil	Nitrogen in the soil, essential for fertility	kg
herbaceous_type	HerbaceousCover	Type of herbaceous cover	(Crop, natural prairie, cultivated prairie)
tree_count	TreeCover	Number of trees	None
seasonal_fodder	HerbaceousCover	Seasonal Fodder	None
young_male_count	Herd	Young Male Count	None
young_female_count	Herd	Young Female Count	None
sub_adult_male_count	Herd	Sub-Adult Male Count	None
sub_adult_female_count	Herd	Sub-Adult Female Count	None
adult_male_count	Herd	Adult Male Count	None
adult_female_count	Herd	Adult Female Count	None

Le processus agit sur les variables suivantes :

Variables d'état	Entité	Définition	Unité
seasonal_fodder	HerbaceousCover	Seasonal Fodder	None
soil_nitrogen	Soil	Nitrogen in the soil, essential for fertility	kg

et dépend des paramètres suivants :

Paramètres	Entité	Définition	Unité
fodder_consumption_rate	Parameter	Fodder Consumption Rate	kgDM/animal/day
herbaceous_growth_rate	Parameter	Herbaceous Growth Rate	kgDM/m²/step
rainfall_normal	Parameter	Rainfall Normal	mm
fodder_quality	Parameter	Fodder Quality	index
maximum_grazing_capacity	Parameter	Maximum Grazing Capacity	TLU/ha
herbaceous_n_uptake_ratio	Parameter	Nitrogen uptake ratio by herbaceous	kgN/kgDM

Paramètres	Entité	Définition	Unité
n_opt	Parameter	Optimal soil nitrogen for plant growth	kgN/ha

Comportement

Description détaillée du processus (équations, états, activités, etc.).

Équation :

```
Growth = r_max * min(1, Rainfall / Rain_opt) * min(1, SoilN / N_opt) * Stress * Quality
```

Pseudo-code

Pseudo-code tel que fourni par le modèle.

```
 // 1. Get Environmental Limiting Factors
optimal_rainfall = optimal_rainfall /* property_031 */;
rainfall = rainfall /* resource_010 */;
rain_factor = min(1, rainfall / optimal_rainfall);

soil_nitrogen_current = soil_nitrogen /* resource_001 */;
n_factor = min(1, soil_nitrogen_current / n_opt) /* resource_001 */ /* property_097 */;

// 2. Calculate Grazing Pressure (Stress Factor)
fodder_consumption_rate = fodder_consumption_rate /* property_060 */;
total_demand = (young_male_count + young_female_count +
                sub_adult_male_count + sub_adult_female_count +
                adult_male_count + adult_female_count) * fodder_consumption_rate;

max_grazing_capacity = max_grazing_capacity /* #property_073 */;
seasonal_fodder_current = seasonal_fodder /* resource_041 */;

if (seasonal_fodder_current > 0) {
  // Calculate pressure ratio. Denominator is capped at max(1, ...) to prevent division by zero
  grazing_pressure = min(1.0, total_demand / max(1, seasonal_fodder_current * max_grazing_capacity));
} else {
  // Maximum pressure (1.0) when no fodder is available
  grazing_pressure = 1.0;
}

// Stress is the reduction factor (0-1) where 1 is no stress and 0 is max stress
stress = max(0, 1 - grazing_pressure);

// 3. Calculate Final Growth
r_max = herbaceous_growth_rate /* property_039 */;
fodder_quality = fodder_quality /* property_053 */;

growth = r_max * rain_factor * n_factor * stress * fodder_quality;

// 4. Update Resources (Mass Balance)
seasonal_fodder = seasonal_fodder + growth;

// Nitrogen Uptake (Robustness: Ensure Soil_Nitrogen does not go below zero)
n_uptake_ratio = herbaceous_n_uptake_ratio /* property_096 */;
n_consumption_needed = growth * n_uptake_ratio;

n_deducted = min(n_consumption_needed, soil_nitrogen_current); // Deduct only what's available
soil_nitrogen = soil_nitrogen - n_deducted;
```

Entité : Crop (agent_007)

Documentation

Crops cultivated by households

Attributs

Les ressources détenues par l'agent :

Paramètre	Commentaire	Unité
`crop_residue`	Crop Residue	None
`forage_crop_stock`	Estimated of harvested forage crop cultivated by the household for supplementary animal feeding.	kg

Dynamiques

Processus: crop_production

Documentation :

Seasonal production of crop yield and crop residue, limited by environment.

Attributs

Le processus dépend des variables suivantes :

Variables d'état	Entité	Définition	Unité
rainfall	ClimateSystem	Rainfall	mm/year
soil_nitrogen	Soil	Nitrogen in the soil, essential for fertility	kg
crop_residue	AgroPastoralHousehold	Crop Residue	None
crop_yield	AgroPastoralHousehold	The mass of harvested crop grain/product.	kg
sam_to_slaughter	Slaughterhouse	Count of Sub-Adult Male animals transferred to the slaughterhouse	Count
property_065	...	...	...

Le processus agit sur les variables suivantes :

Variables d'état	Entité	Définition	Unité
crop_yield	AgroPastoralHousehold	The mass of harvested crop grain/product.	kg
crop_residue	AgroPastoralHousehold	Crop Residue	None
soil_nitrogen	Soil	Nitrogen in the soil, essential for fertility	kg

et dépend des paramètres suivants :

Paramètres	Entité	Définition	Unité
crop_yield_rate_r_max	Parameter	Maximum intrinsic proportional growth rate of crop biomass per season.	1/step
harvest_index	Parameter	Proportion of total crop biomass that is marketable grain/yield.	proportion
rainfall_normal	Parameter	Rainfall Normal	mm

Paramètres	Entité	Définition	Unité
n_opt	Parameter	Optimal soil nitrogen for plant growth	kgN/ha
crop_n_uptake_ratio	Parameter	Proportional mass of Nitrogen consumed per unit of crop biomass growth.	kgN/kgDM

Comportement

Description détaillée du processus (équations, états, activités, etc.).

Équation :

```
TotalBiomass = r_max * Area * f(Rain) * f(N)
```

Pseudo-code

Pseudo-code tel que fourni par le modèle.

```
 // 1. Determine Limiting Factors
r_max = crop_yield_rate_r_max /* property_120 */;
total_crop_area = crop_area /* property_065 */;

// Get current resource values
rainfall_current = rainfall /* resource_010 */;
soil_nitrogen_current = soil_nitrogen /* resource_001 */;
optimal_rainfall = optimal_rainfall /* property_031 */;
n_opt = n_opt /* property_097 */;

// Environmental factors
rain_factor = min(1, rainfall_current / optimal_rainfall);
n_factor = min(1, soil_nitrogen_current / n_opt);

// 2. Calculate Total Biomass Produced
total_biomass = r_max * total_crop_area * rain_factor * n_factor;

// 3. Partition Biomass into Yield and Residue (Harvest Index)
harvest_index = harvest_index /* property_121 */;

crop_yield_produced = total_biomass * harvest_index;
crop_residue_produced = total_biomass * (1 - harvest_index);

// 4. Calculate Nitrogen Uptake (Consumption with Robustness Check)
n_uptake_ratio = crop_n_uptake_ratio /* property_050 */;
n_consumed_needed = total_biomass * n_uptake_ratio;

// The amount deducted cannot exceed the current stock
n_consumed_actual = min(n_consumed_needed, soil_nitrogen_current);

// 5. Update Resources (Mass Balance)
// A. Add Residue to the stock (held by Crops Agent)
crop_residue /* resource_035 */ = crop_residue + crop_residue_produced;

// B. Add Yield
crop_yield /* resource_028 */ = crop_yield + crop_yield_produced;

// C. Consume Soil Nitrogen (Mass Balance)
soil_nitrogen /* resource_001 */ = soil_nitrogen - n_consumed_actual;
```

Processus: crop_residue_partitioning

Documentation :

Management of crop residue: partitioning into animal feed and cultivated litter.

Attributs

Le processus dépend des variables suivantes :

Variables d'état	Entité	Définition	Unité
crop_residue	AgroPastoralHousehold	Crop Residue	None
crop_residue_to_feed	AgroPastoralHousehold	Residue to feed by herd	None
cultivated_litter	Soil	Cultivated litter	None

Le processus agit sur les variables suivantes :

Variables d'état	Entité	Définition	Unité
crop_residue	AgroPastoralHousehold	Crop Residue	None
crop_residue_to_feed	AgroPastoralHousehold	Residue to feed by herd	None
cultivated_litter	Soil	Cultivated litter	None

et dépend des paramètres suivants :

Paramètres	Entité	Définition	Unité
residue_grazing_prop	Parameter	Proportion of crop residue consumed by animals after harvest.	proportion
residue_incorp_prop	Parameter	Proportion of crop residue incorporated into the soil (Cultivated Litter).	proportion

Comportement

Description détaillée du processus (équations, états, activités, etc.).

Équation :

`ResidueConsumed = ResidueStock * (GrazingProp + IncorpProp)` `Feed = Residue * GrazingProp` `Litter = Residue * IncorpProp`

Pseudo-code

Pseudo-code tel que fourni par le modèle.

```
 // Total residue available
residue_stock = crop_residue /* resource_035 */;

// Get Proportions
grazing_prop = residue_grazing_prop /* property_124 */;
incorporation_prop = residue_incorp_prop /* property_125 */;

// 1. Conditional Execution and Proportion Scaling (Robustness Check)
if (residue_stock > 0) {
  // Calculate total requested consumption proportion
  total_management_prop = grazing_prop + incorporation_prop;

  if (total_management_prop > 1.0) {
    // Scale proportions down so their sum equals 1.0, consuming all residue
    grazing_prop_actual = grazing_prop / total_management_prop;
    incorporation_prop_actual = incorporation_prop / total_management_prop;
  } else {
    // Use original proportions (leaving unallocated residue in stock)
    grazing_prop_actual = grazing_prop;
    incorporation_prop_actual = incorporation_prop;
  }

  // 2. Calculate final allocations
  residue_to_feed = residue_stock * grazing_prop_actual;
  residue_to_litter = residue_stock * incorporation_prop_actual;

  // Total mass consumed from stock
  total_residue_consumed = residue_to_feed + residue_to_litter;

  // 3. Update Resources (Mass Balance)
  // Consume the stock (resource_035)
  crop_residue /* resource_035 */ = crop_residue - total_residue_consumed;

  // Increase destination stocks
  crop_residue_to_feed /* resource_033 */ = crop_residue_to_feed + residue_to_feed;
  cultivated_litter /* resource_008 */ = cultivated_litter + residue_to_litter;
} else {
  // No action if no residue is available
  residue_to_feed = 0;
  residue_to_litter = 0;
}
```

Entité : Atmosphere (agent_004)

Documentation

Atmosphere

Attributs

Les ressources détenues par l'agent :

Paramètre	Commentaire	Unité
`atmospheric_nitrogen`	Atmospheric nitrogen	kg

Paramètre	Commentaire	Unité
`ghg`	Greenhouse Gases	None

Dynamiques

Section vide : à compléter manuellement

Entité : AgroPastoralHousehold (agent_005)

Documentation

Unified household agent managing integrated farming and herding activities (Agro-Pastoralism). The central decision-making unit for livelihood strategies and resilience.

Attributs

Les ressources détenues par l'agent :

Paramètre	Commentaire	Unité
`cash`	Revenue	None
`milk`	Milk	None
`milk_for_consumption`	Stock of milk retained by the household for direct nutritional use, separate from milk sold or left for calves.	L
`milk_for_sale`	Stock of milk sold by the household for direct consumption or for dairy	L
`fertilizer_stock`	Stock of fertilizer available to the household for use in crop production (forage or cereals).	None
`crop_yield`	The mass of harvested crop grain/product.	kg
`hay`	Stored, conserved seasonal fodder used as supplementary feed.	None
`concentrate`	Concentrate	None
`crop_residue_to_feed`	Residue to feed by herd	None
`traditional_knowledge`	Traditional Knowledge	None
`available_labor`	The total labor capacity (e.g., person-hours) available to the Households agent.	person-hours
`crop_area`	The total area under cultivation by the Households agent.	ha
`off_farm_income`	Cash earned from off-farm or non-pastoral labor activities.	None

Paramètre	Commentaire	Unité
`livestock_revenue`	Total cash generated from the sale of livestock (live animals and milk).	None
`crop_sale_revenue`	Total cash generated from the sale of crop yield (e.g., cereals, legumes).	None
`herd_tlu`	The aggregated livestock population of the Agro-Pastoral Households, converted to Tropical Livestock Units (TLU).	None
`seasonal_fodder_reliance`	The proportion of the herd's required feed intake that must be sourced from external Seasonal Fodder, indicating pressure to migrate.	dimensionless

Dynamiques

Processus: feeding_and_consumption

Documentation :

Livestock feeding and consumption process

Attributs

Le processus dépend des variables suivantes :

Variables d'état	Entité	Définition	Unité
seasonal_fodder	HerbaceousCover	Seasonal Fodder	None
hay	AgroPastoralHousehold	Stored, conserved seasonal fodder used as supplementary feed.	None
crop_residue_to_feed	AgroPastoralHousehold	Residue to feed by herd	None
concentrate	AgroPastoralHousehold	Concentrate	None
young_male_count	Herd	Young Male Count	None
young_female_count	Herd	Young Female Count	None
sub_adult_male_count	Herd	Sub-Adult Male Count	None
sub_adult_female_count	Herd	Sub-Adult Female Count	None
adult_male_count	Herd	Adult Male Count	None
adult_female_count	Herd	Adult Female Count	None
body_weight	Herd	Body Weight (live weight (LW))	None

Variables d'état	Entité	Définition	Unité
seasonal_fodder_reliance	AgroPastoralHousehold	The proportion of the herd's required feed intake that must be sourced from external Seasonal Fodder, indicating pressure to migrate.	dimensionless

Le processus agit sur les variables suivantes :

Variables d'état	Entité	Définition	Unité
nutrient_consumed	Herd	Nutrient consumed by herd	mg/kg
daily_intake_ratio	Herd	The ratio of Actual Fodder Intake to Required Fodder Intake (from process_003). Key driver for Body Condition.	dimensionless
seasonal_fodder	HerbaceousCover	Seasonal Fodder	None
hay	AgroPastoralHousehold	Stored, conserved seasonal fodder used as supplementary feed.	None
crop_residue_to_feed	AgroPastoralHousehold	Residue to feed by herd	None
concentrate	AgroPastoralHousehold	Concentrate	None
seasonal_fodder_reliance	AgroPastoralHousehold	The proportion of the herd's required feed intake that must be sourced from external Seasonal Fodder, indicating pressure to migrate.	dimensionless

et dépend des paramètres suivants :

Paramètres	Entité	Définition	Unité
intake_factor	Parameter	Intake Factor	index
body_weight	Parameter	Body Weight	kg/animal
mobility_need	Parameter	Need for mobility based on resource availability	MJ/km
metabolic_need	Parameter	Metabolic Need	MJ/kg_liveweight/day
fodder_consumption_rate	Parameter	Fodder Consumption Rate	kgDM/animal/day
fodder_quality	Parameter	Fodder Quality	index
lactation_need	Parameter	Lactation Need	index
feed_nutrient_content	Parameter	The average nutrient/energy density of the total consumed feed mix (DM, Hay, Concentrate, Residue).	MJ/kgDM

Comportement

Description détaillée du processus (équations, états, activités, etc.).

Équation :

`NutrientConsumed = ActualIntake * FeedNutrientContent` `ActualIntake = min(RequiredFodder, AdjustedAvailableFodder)`

Pseudo-code

Pseudo-code tel que fourni par le modèle.

```
 // 1. Determine Required Fodder (Dry Matter Intake Requirement)
total_animals = young_male_count /* resource_019 */
              + young_female_count /* resource_021 */
              + sub_adult_male_count /* resource_022 */
              + sub_adult_female_count /* resource_023 */
              + adult_male_count /* resource_024 */
              + adult_female_count /* resource_025 */;

if (total_animals > 0) {
  default_body_weight = body_weight_default /* property_052 */;
  herd_biomass = total_animals * default_body_weight;
  fodder_consumption_rate = fodder_consumption_rate /* property_060 */;

  // Adjust requirement based on other needs (Metabolic, Mobility, Lactation)
  metabolic_need = metabolic_need /* property_019 */;
  mobility_need = mobility_need /* property_020 */;
  lactation_need = lactation_need /* property_063 */;

  // Total required fodder (DM) including needs adjustments
  required_fodder_dm = herd_biomass * fodder_consumption_rate;
  required_fodder_dm = required_fodder_dm * (1 + metabolic_need + mobility_need + lactation_need);

  // 2. Hierarchical Consumption (Private Stock Priority)

  // Available Private Stocks (The stock that is consumed FIRST)
  hay_stock = hay /* resource_032 */;
  residue_stock = crop_residue_to_feed /* resource_033 */;
  concentrate_stock = concentrate /* resource_034 */;

  // Total usable private stock for feeding (assuming 80% usable fraction for concentrate)
  total_private_stock = hay_stock + residue_stock + (concentrate_stock * 0.8);

  // Amount covered by private stock
  consumed_from_private = min(required_fodder_dm, total_private_stock);

  // Remaining deficit must be covered by external Seasonal Fodder (rangelands)
  deficit_to_be_covered = required_fodder_dm - consumed_from_private;

  // 3. External Fodder Reliance and Actual Intake

  // Fodder quality and intake factor applied to the *deficit* (as external fodder is typically lower
  quality_factor = max(0.0, min(1.0, fodder_quality /* property_053 */));
  intake_mod_factor = max(0.0, intake_factor /* property_051 */);

  seasonal_fodder_stock = seasonal_fodder /* resource_041 */;

  // Adjusted capacity of seasonal fodder to cover the deficit
  adjusted_seasonal_fodder = seasonal_fodder_stock * quality_factor * (1 + intake_mod_factor);
```

```
  // Actual mass consumed from seasonal fodder (cannot exceed stock or adjusted capacity)
  consumed_from_seasonal = min(deficit_to_be_covered, adjusted_seasonal_fodder);

  // Total actual intake (DM)
  actual_intake_dm = consumed_from_private + consumed_from_seasonal;

  // 4. Mass Balance and Migration Pressure Output

  // A. Consume Private Stocks (Proportional Cut)
  if (total_private_stock > 1e-6) {
    consumption_ratio_private = consumed_from_private / total_private_stock;
    hay /* resource_032 */ = hay - hay_stock * consumption_ratio_private;
    crop_residue_to_feed /* resource_033 */ = crop_residue_to_feed - residue_stock * consumption_rati
    // Convert concentrate back to full stock consumption
    concentrate /* resource_034 */ = concentrate - (concentrate_stock * 0.8) * consumption_ratio_priv
  }

  // B. Consume Seasonal Fodder
  seasonal_fodder /* resource_041 */ = seasonal_fodder - consumed_from_seasonal;

  // C. Migration Pressure Output (The link to process_009)
  // Ratio of total requirement that HAD to be sourced from external seasonal fodder
  seasonal_fodder_reliance = min(1.0, deficit_to_be_covered / max(required_fodder_dm, 1e-6));
  seasonal_fodder_reliance /* resource_073 */ = seasonal_fodder_reliance;

  // D. Nutrient Mass and Animal Status Updates
  feed_nutrient_content = feed_nutrient_content /* property_106 */;
  nutrients_consumed_mass = actual_intake_dm * feed_nutrient_content;
  nutrients_consumed /* resource_051 */ = nutrients_consumed_mass;

  // Update daily_intake_ratio (Input for process_008)
  daily_intake_ratio /* resource_075 */ = actual_intake_dm / max(required_fodder_dm, 1e-6);
} else {
  nutrients_consumed /* resource_051 */ = 0;
  seasonal_fodder_reliance /* resource_073 */ = 0;
  daily_intake_ratio /* resource_075 */ = 0;
}
```

Processus: milking

Documentation :

Milk extraction from lactating females

Attributs

Le processus dépend des variables suivantes :

Variables d'état	Entité	Définition	Unité
adult_female_count	Herd	Adult Female Count	None
livestock_health	Herd	Livestock Health	None
body_condition	Herd	Body Condition (body condition score (BCS))	None
off_farm_income	AgroPastoralHousehold	Cash earned from off-farm or non-pastoral labor activities.	None

Le processus agit sur les variables suivantes :

Variables d'état	Entité	Définition	Unité
milk	AgroPastoralHousehold	Milk	None
off_farm_income	AgroPastoralHousehold	Cash earned from off-farm or non-pastoral labor activities.	None
milk	AgroPastoralHousehold	Milk	None

et dépend des paramètres suivants :

Paramètres	Entité	Définition	Unité
base_milk_yield	Parameter	Average milk yield (L/day/animal) under optimal conditions.	L/animal/day
min_yield_threshold	Parameter	Minimum Yield Threshold	kgDM/ha
max_yield_threshold	Parameter	Maximum Yield Threshold	kgDM/ha
total_lactation_period	Parameter	The total number of days in the lactation cycle.	days
lactating_proportion	Parameter	The average proportion of Adult Female Count that is currently lactating.	proportion
optimal_condition	Parameter	The Body Condition Score (BCS) value representing optimal health and body reserves, used as a reference for carcass yield and stress calculation.	BCS
days_lactation_avg	Parameter	The average number of days the lactating females in the herd have been in milk.	days
labor_rate_milking	Parameter	The amount of labor consumed (e.g., man-days/head) required per lactating female to perform milking.	man-day/head/step

Comportement

Description détaillée du processus (équations, états, activités, etc.).

Équation :

```
MilkYield = BaseMilk * NumLactating * LactationCurve * NutritionFactor * HealthFactor
```

Pseudo-code

Pseudo-code tel que fourni par le modèle.

```
 // Get parameters
base_yield = base_milk_yield /* property_070 */;
max_yield = maximum_yield /* property_091 */;
min_yield = minimum_yield /* property_090 */;
total_lactation_period_tmp = total_lactation_period /* property_071 */;

// 1. Determine number of lactating females (needs an external counter resource)
lactating_prop = lactating_proportion /* property_115 */;
num_lactating = adult_female_count * lactating_prop /* resource_025 */;

// 2. Calculate Modifiers (DYNAMIC FACTORS)
days_in_lactation_tmp = days_lactation_avg /* property_116 */;
stage = days_in_lactation_tmp / total_lactation_period_tmp;
curve = 1 - (stage * stage); // Decreasing curve (Lactation Curve Factor)

// Nutrition Factor: Ratio of current Body Condition to optimal
nutrition_factor = body_condition / optimal_condition /* resource_046 property_114 */;

// Health Factor: Directly use Livestock Health resource
health_factor = livestock_health /* resource_040 */;

// 3. Calculate Total Milk Yield
total_milk = base_yield * num_lactating * curve * nutrition_factor * health_factor;

// 4. Apply yield limits
if (total_milk < min_yield * num_lactating) {
  total_milk = min_yield * num_lactating;
}
total_milk = min(total_milk, max_yield * num_lactating);

// 5. Calculate Labor Consumed
labor_consumed = num_lactating * labor_rate_milking /* property_034 */;

// 6. Update Resources (Mass Balance)
milk /* resource_039 */ = milk + total_milk;
available_labor /* resource_066 */ = available_labor - labor_consumed;
```

Processus: concentrate_purchase

Documentation :

Purchase of concentrate by AgroPastoralHouseholds from Markets.

Attributs

Le processus dépend des variables suivantes :

Variables d'état	Entité	Définition	Unité
cash	AgroPastoralHousehold	Revenue	None
concentrate	AgroPastoralHousehold	Concentrate	None
available_concentrate_stock	Markets	Available Concentrate Stock	None

Le processus agit sur les variables suivantes :

Variables d'état	Entité	Définition	Unité
cash	AgroPastoralHousehold	Revenue	None
concentrate	AgroPastoralHousehold	Concentrate	None
available_concentrate_stock	Markets	Available Concentrate Stock	None

et dépend des paramètres suivants :

Paramètres	Entité	Définition	Unité
concentrate_price	Parameter	Concentrate Price	currency/kg
proportion	Parameter	Proportion	proportion

Comportement

Description détaillée du processus (équations, états, activités, etc.).

Équation :

`Cost = Qty * PriceConc` `ΔConcentrate = +Qty` `ΔAvailableStock = -Qty` `ΔCash = -Cost`

Pseudo-code

Pseudo-code tel que fourni par le modèle.

```
 // Calculate Cost and quantity
purchased_qty = quantity /* property_025 */;
cost = purchased_qty * concentrate_price /* property_022 */;

// Decision: Check affordability and market stock availability
if (cash >= cost and available_concentrate_stock >= purchased_qty) {
  // Execute Purchase and Transfer

  // 1. Decrease AgroPastoralHouseholds Cash
  cash /* resource_014 */ = cash - cost;

  // 2. Decrease Market Stock
  available_concentrate_stock /* resource_038 */ = available_concentrate_stock + purchased_qty;

  // 3. Increase AgroPastoralHouseholds Stock
  concentrate /* resource_034 */ =  concentrate + purchased_qty;
} else {
  // Purchase failed (not enough cash or market stock)
  purchased_qty = 0;
}
```

Processus: animal_sale

Documentation :

Commercial sale of live animals by AgroPastoralHouseholds, generating cash revenue.

Attributs

Le processus dépend des variables suivantes :

Variables d'état	Entité	Définition	Unité
young_male_count	Herd	Young Male Count	None
young_female_count	Herd	Young Female Count	None
sub_adult_male_count	Herd	Sub-Adult Male Count	None
sub_adult_female_count	Herd	Sub-Adult Female Count	None
adult_male_count	Herd	Adult Male Count	None
adult_female_count	Herd	Adult Female Count	None
livestock_revenue	AgroPastoralHousehold	Total cash generated from the sale of livestock (live animals and milk).	None

Le processus agit sur les variables suivantes :

Variables d'état	Entité	Définition	Unité
livestock_revenue	AgroPastoralHousehold	Total cash generated from the sale of livestock (live animals and milk).	None
young_male_count	Herd	Young Male Count	None
young_female_count	Herd	Young Female Count	None
sub_adult_male_count	Herd	Sub-Adult Male Count	None
sub_adult_female_count	Herd	Sub-Adult Female Count	None
adult_male_count	Herd	Adult Male Count	None
adult_female_count	Herd	Adult Female Count	None
ym_to_slaughter	Slaughterhouse	Count of Young Male animals transferred to the slaughterhouse	Count
yf_to_slaughter	Slaughterhouse	Count of Young Female animals transferred to the slaughterhouse	Count
sam_to_slaughter	Slaughterhouse	Count of Sub-Adult Male animals transferred to the slaughterhouse	Count
saf_to_slaughter	Slaughterhouse	Count of Sub-Adult Female animals transferred to the slaughterhouse	Count
am_to_slaughter	Slaughterhouse	Count of Adult Male animals transferred to the slaughterhouse	Count
af_to_slaughter	Slaughterhouse	Count of Adult Female animals transferred to the slaughterhouse	Count

et dépend des paramètres suivants :

Paramètres	Entité	Définition	Unité
offtake_adult_male	Parameter	Offtake Adult Male	proportion/step
offtake_adult_female	Parameter	Offtake Adult Female	proportion/step
offtake_sub_adult_male	Parameter	Offtake Sub-Adult Male	proportion/step
offtake_sub_adult_female	Parameter	Offtake Sub-Adult Female	proportion/step
offtake_young_male	Parameter	Offtake Young Male	proportion/step
offtake_young_female	Parameter	Offtake Young Female	proportion/step
live_animal_price	Parameter	The average price received per animal (or per unit of weight) sold to the market/slaughterhouse.	currency/animal
slaughter_prop_young_male	Parameter	Fraction of Young Males sold that are destined for the slaughterhouse.	proportion
slaughter_prop_young_female	Parameter	Fraction of Young Females sold that are destined for the slaughterhouse.	proportion
slaughter_prop_sub_adult_male	Parameter	Fraction of Sub-Adult Males sold that are destined for the slaughterhouse.	proportion
slaughter_prop_sub_adult_female	Parameter	Fraction of Sub-Adult Females sold that are destined for the slaughterhouse.	proportion
slaughter_prop_adult_male	Parameter	Fraction of Adult Males sold that are destined for the slaughterhouse.	proportion
slaughter_prop_adult_female	Parameter	Fraction of Adult Females sold that are destined for the slaughterhouse.	proportion

Comportement

Description détaillée du processus (équations, états, activités, etc.).

Équation :

`For each class i: NumToSell_i = Count_i * OfftakeRate_i` `ΔCash = Σ(NumToSell_i * LivePrice)`

Pseudo-code

Pseudo-code tel que fourni par le modèle.

```
 // Initialize revenue
sold_amount = 0;
live_price = live_animal_price /* property_107 */;

// Calculate sales and slaughter flows per class (explicit, no loop)

// Young Male
to_sell_ym = round(young_male_count /* resource_019 */ * offtake_young_male /* property_067 */);
```

```
if (to_sell_ym > 0) {
  sold_amount = sold_amount +(to_sell_ym * live_price);
  young_male_count /* resource_019 */ = young_male_count - to_sell_ym;
  ym_to_slaughter /* resource_054 */ = round(to_sell_ym * slaughter_prop_ym /* property_108 */);
} else {
  ym_to_slaughter /* resource_054 */ = 0;
}

// Young Female
to_sell_yf = round(young_female_count /* resource_021 */ * offtake_young_female /* property_068 */);
if (to_sell_yf > 0) {
  sold_amount = sold_amount + (to_sell_yf * live_price);
  young_female_count /* resource_021 */ = young_female_count - to_sell_yf;
  yf_to_slaughter /* resource_055 */ = round(to_sell_yf * slaughter_prop_yf /* property_109 */);
} else {
  yf_to_slaughter /* resource_055 */ = 0;
}

// Sub-Adult Male
to_sell_sam = round(sub_adult_male_count /* resource_022 */ * offtake_sub_adult_male /* property_069
if (to_sell_sam > 0) {
  sold_amount = sold_amount + (to_sell_sam * live_price);
  sub_adult_male_count /* resource_022 */ = sub_adult_male_count - to_sell_sam;
  sam_to_slaughter /* resource_056 */ = round(to_sell_sam * slaughter_prop_sam /* property_110 */);
} else {
  sam_to_slaughter /* resource_056 */ = 0;
}

// Sub-Adult Female
to_sell_saf = round(sub_adult_female_count /* resource_023 */ * offtake_sub_adult_female /* property_
if (to_sell_saf > 0) {
  sold_amount = sold_amount + (to_sell_saf * live_price);
  sub_adult_female_count /* resource_023 */ = sub_adult_female_count - to_sell_saf;
  saf_to_slaughter /* resource_057 */ = round(to_sell_saf * slaughter_prop_saf /* property_111 */);
} else {
  saf_to_slaughter /* resource_057 */ = 0;
}

// Adult Male
to_sell_am = round(adult_male_count /* resource_024 */ * offtake_adult_male /* property_006 */);
if (to_sell_am > 0) {
  sold_amount = sold_amount + (to_sell_am * live_price);
  adult_male_count /* resource_024 */ = adult_male_count - to_sell_am;
  am_to_slaughter /* resource_058 */ = round(to_sell_am * slaughter_prop_am /* property_112 */);
} else {
  am_to_slaughter /* resource_058 */ = 0;
}

// Adult Female
to_sell_af = round(adult_female_count /* resource_025 */ * offtake_adult_female /* property_007 */);
if (to_sell_af > 0) {
  sold_amount = sold_amount + (to_sell_af * live_price);
  adult_female_count /* resource_025 */ = adult_female_count - to_sell_af;
  af_to_slaughter /* resource_059 */ = round(to_sell_af * slaughter_prop_af /* property_113 */);
} else {
  af_to_slaughter /* resource_059 */ = 0;
}

// Update Household's Cash (Revenue from live sales)
revenue /* resource_067 */ = revenue + sold_amount;
```

Processus: migration

Documentation :

Long-distance seasonal migration of pastoral herds

Attributs

Le processus dépend des variables suivantes :

Variables d'état	Entité	Définition	Unité
young_male_count	Herd	Young Male Count	None
young_female_count	Herd	Young Female Count	None
sub_adult_male_count	Herd	Sub-Adult Male Count	None
sub_adult_female_count	Herd	Sub-Adult Female Count	None
adult_male_count	Herd	Adult Male Count	None
adult_female_count	Herd	Adult Female Count	None
seasonal_fodder_reliance	AgroPastoralHousehold	The proportion of the herd's required feed intake that must be sourced from external Seasonal Fodder, indicating pressure to migrate.	dimensionless
water	WaterSource	Water	None
rainfall	ClimateSystem	Rainfall	mm/year

Le processus agit sur les variables suivantes :

Variables d'état	Entité	Définition	Unité
body_weight	Herd	Body Weight (live weight (LW))	None
body_condition	Herd	Body Condition (body condition score (BCS))	None

et dépend des paramètres suivants :

Paramètres	Entité	Définition	Unité
...	...	...	...

Comportement

Description détaillée du processus (équations, états, activités, etc.).

Équation :

```
MigrationDecision = ResourceScarcity * ClimateStress * TraditionalTiming
```

Pseudo-code

Pseudo-code tel que fourni par le modèle.

```
 // 1. Retrieve Pre-calculated Decision Drivers

// A. Fodder Scarcity (DIRECT LINK TO FEEDING PROCESS)
// If Reliance is 1, household MUST migrate to find seasonal fodder.
seasonal_fodder_reliance = seasonal_fodder_reliance /* resource_073 */;
fodder_scarcity = seasonal_fodder_reliance;

// B. Water Scarcity
// Compute herd biomass (kg) from class counts
total_animals = young_male_count /* resource_019 */
              + young_female_count /* resource_021 */
              + sub_adult_male_count /* resource_022 */
              + sub_adult_female_count /* resource_023 */
              + adult_male_count /* resource_024 */
              + adult_female_count /* resource_025 */;
herd_biomass = total_animals * body_weight_default /* property_052 */;

current_water = seasonal_water /* resource_042 */ + rainfall /* resource_010 */;
required_water = herd_biomass * water_consumption_rate /* property_061 */;

if (required_water > 1e-6) {
  water_status = current_water / required_water;
  water_scarcity = 1 - min(1.0, water_status);
} else {
  water_scarcity = 0; // No required water means no scarcity
}

// Overall Resource Scarcity
resource_scarcity = max(fodder_scarcity, water_scarcity); // Use the highest pressure point

// 2. Environmental stress (Climate)
// (Calculations remain simplified and rely on external functions/properties for details)
rainfall_variability = min(1.5, max(0.5, rainfall /* resource_010 */ / rainfall_normal /* property_03
temperature_factor = min(1, temperature /* resource_013 */ / temp_opt /* property_099 */);
climate_stress = rainfall_variability * temperature_factor;

// 3. Traditional knowledge influence
seasonal_timing = saison /* property_098 */;

// 4. Decision making: Calculate Migration Pressure
migration_pressure = resource_scarcity * climate_stress * seasonal_timing;

movement_threshold = movement_threshold /* property_049 */;

if (migration_pressure > movement_threshold) {
  // Plan migration
  min_herd_condition = min_herd_condition /* property_100 */;
  current_herd_condition = herd_condition /* resource_046 */;

  // Execute movement is contingent on herd health and minimum condition
  if (current_herd_condition > min_herd_condition) {
    move_herd(destination);
    update_resource_access(); // New access to external Seasonal Fodder is granted
    adapt_herd_management();
    update_herd_condition();
    update_resource_use();
  }
}
```

Processus: meat_sale

Documentation :

Sale of processed meat stock, tracking the cumulative volume sold.

Attributs

Le processus dépend des variables suivantes :

Variables d'état	Entité	Définition	Unité
am_to_slaughter	Slaughterhouse	Count of Adult Male animals transferred to the slaughterhouse	Count
meat_sold_amount	Slaughterhouse	The total mass (kg) of processed meat sold to the market over the simulation period.	kg

Le processus agit sur les variables suivantes :

Variables d'état	Entité	Définition	Unité
am_to_slaughter	Slaughterhouse	Count of Adult Male animals transferred to the slaughterhouse	Count
meat_sold_amount	Slaughterhouse	The total mass (kg) of processed meat sold to the market over the simulation period.	kg

et dépend des paramètres suivants :

Paramètres	Entité	Définition	Unité
market_price_meat	Parameter	Market Price for meat after all processing in slaughterhouse	currency/kg

Comportement

Description détaillée du processus (équations, états, activités, etc.).

Équation :

`MeatSoldCum += ProcessedMeatStock` `DeltaStock = -ProcessedMeatStock`

Pseudo-code

Pseudo-code tel que fourni par le modèle.

```
 // quantity to sell is the current stock
to_sell = processed_meat_stock /* resource_058 */;
market_price = market_price_meat /* property_030 */;

if (to_sell > 0) {
  // Calculate revenue (The financial transaction is abstracted away, but revenue is calculated for c
  revenue = to_sell * market_price;

  // Execute transaction: Decrease Stock and Update Cumulative Metric

  // 1. Decrease Stock (consumed)
  processed_meat_stock = processed_meat_stock - to_sell;  /* resource_058 */

  // 2. Update Cumulative Meat Sold Amount
  Meat_Sold_Amount_Cumulative = Meat_Sold_Amount_Cumulative + to_sell /* resource_061 */;
 }
else {
  // No stock to sell, no change
  revenue = 0;
  }
```

Processus: milk_sale

Documentation :

Splitting of produced milk into direct market sales and transfer to Dairy for processing.

Attributs

Le processus dépend des variables suivantes :

Variables d'état	Entité	Définition	Unité
milk_for_sale	AgroPastoralHousehold	Stock of milk sold by the household for direct consumption or for dairy	L
cash	AgroPastoralHousehold	Revenue	None
dairy_milk	Dairy	Raw milk transferred from the AgroPastoralHouseholds for dairy processing.	Liters
milk_for_consumption	AgroPastoralHousehold	Stock of milk retained by the household for direct nutritional use, separate from milk sold or left for calves.	L

Le processus agit sur les variables suivantes :

Variables d'état	Entité	Définition	Unité
milk_for_sale	AgroPastoralHousehold	Stock of milk sold by the household for direct consumption or for dairy	L
livestock_revenue	AgroPastoralHousehold	Total cash generated from the sale of livestock (live animals and milk).	None

Variables d'état	Entité	Définition	Unité
dairy_milk	Dairy	Raw milk transferred from the AgroPastoralHouseholds for dairy processing.	Liters

et dépend des paramètres suivants :

Paramètres	Entité	Définition	Unité
milk_dairy_prop	Parameter	The fractional of sold milk for dairy	proportion
market_price_raw_milk	Parameter	The market price per unit (kg or L) of the milk.	currency/L

Comportement

Description détaillée du processus (équations, états, activités, etc.).

Équation :

`MilkSold = Milk * SaleProp` `MilkToDairy = Milk * (1 - SaleProp)` `Revenue = MilkSold * PriceLait`

Pseudo-code

Pseudo-code tel que fourni par le modèle.

```
 // Total raw milk produced by AgroPastoralHousehold
total_milk_sale = milk_for_sale /* resource_071 */;
market_price = market_price_raw_milk /* property_119 */;
dairy_prop = milk_dairy_prop /* property_146 */

if milk_sold > 0:
  // 1. Execute Sale (updates Household's Cash)
  income = total_milk_sale * market_price;
  livestock_revenue = livestock_revenue + income;  /* resource_067 */

  // 2. Execute Transfer to Dairy
  milk_to_dairy = total_milk_sale * dairy_prop
  milk_for_processing = milk_for_processing + milk_to_dairy;  /* resource_029 */

  // 3. Update Resources
  // Consume the raw milk input entirely
  milk_for_sale = milk_for_sale - total_milk; /* resource_039 */
else:
  revenue = 0;
```

Processus: fertilization

Documentation :

Application of synthetic fertilizer to soil, efficiency dependent on soil moisture and pH.

Attributs

Le processus dépend des variables suivantes :

Variables d'état	Entité	Définition	Unité
fertilizer_stock	AgroPastoralHousehold	Stock of fertilizer available to the household for use in crop production (forage or cereals).	None
soil_type	Soil	Type of soil	-
soil_pH	Soil	Soil pH	pH
soil_moisture	Soil	Soil Moisture	%
soil_nitrogen	Soil	Nitrogen in the soil, essential for fertility	kg

Le processus agit sur les variables suivantes :

Variables d'état	Entité	Définition	Unité
fertilizer_stock	AgroPastoralHousehold	Stock of fertilizer available to the household for use in crop production (forage or cereals).	None
soil_nitrogen	Soil	Nitrogen in the soil, essential for fertility	kg

et dépend des paramètres suivants :

Paramètres	Entité	Définition	Unité
application_rate	Parameter	Application Rate	kg/ha/step
max_nitrogen	Parameter	The maximum mineral Nitrogen (kg) the soil can effectively hold/absorb.	kgN/ha
base_efficiency	Parameter	The inherent application efficiency (proportion) under ideal conditions (e.g., 0.6).	proportion
optimal_moisture	Parameter	Soil moisture (fraction) for maximum fertilizer efficiency.	proportion
min_moisture	Parameter	Minimum soil moisture (fraction) required for application.	proportion
fertilizer_to_n	Parameter	Fertilizer to Nitrogen	kgN/kg_fertilizer

Comportement

Description détaillée du processus (équations, états, activités, etc.).

Équation :

`FertApplied = ApplicationRate * Area * Efficiency` `Efficiency = BaseEff * f(Moisture) * f(pH)`

Pseudo-code

Pseudo-code tel que fourni par le modèle.

```
 // Required inputs for logic:
current_soil_nitrogen = soil_nitrogen /* resource_001 */;
fertilizer_stock_current = fertilizer_stock /* resource_015 */;
soil_moisture = soil_moisture /* resource_053 */;
soil_pH = soil_pH /* resource_052 */;

// Parameters
max_nitrogen = max_nitrogen /* property_087 */;
base_efficiency = base_efficiency /* property_088 */;
optimal_moisture = optimal_moisture /* property_089 */;
min_moisture = min_moisture /* property_081 */;
application_rate = application_rate /* property_066 */;

// Check conditions for fertilizer application (must have stock and adequate moisture)
if (soil_moisture > min_moisture and fertilizer_stock_current > 0) {
  // Calculate required N and potential application from stock
  n_needed_by_soil = max(0, max_nitrogen - current_soil_nitrogen);
  potential_application = application_rate;

  // Determine actual amount of fertilizer consumed (limited by what they want to apply and what they
  application_consumed = min(potential_application, fertilizer_stock_current);

  // Calculate efficiency factors
  soil_moisture_factor = min(1, soil_moisture / optimal_moisture);

  // Nested condition for pH effect (écrit de manière explicite)
  if (soil_pH < 6) {
    soil_ph_factor = 0.8;
  } else if (soil_pH > 8) {
    soil_ph_factor = 0.7;
  } else {
    soil_ph_factor = 1.0;
  }

  // Calculate final efficiency
  efficiency = base_efficiency * soil_moisture_factor * soil_ph_factor;

  // Calculate N added to soil
  n_content_factor = fertilizer_to_n /* property_045 */;
  n_added_to_soil_potential = application_consumed * n_content_factor * efficiency;

  // Ensure N_added does not exceed the remaining capacity of the soil
  n_actually_added = min(n_added_to_soil_potential, n_needed_by_soil);

  // Update resources (Mass Balance)
  soil_nitrogen = soil_nitrogen + n_actually_added;
  fertilizer_stock = fertilizer_stock - application_consumed;
} else {
  // Application is zero if moisture is too low or no stock is available
  n_actually_added = 0;
  application_consumed = 0;
}
```

Processus: fertilizer_purchase

Documentation :

Purchase of fertilizer by the household, transferring cash to stock.

Attributs

Le processus dépend des variables suivantes :

Variables d'état	Entité	Définition	Unité
cash	AgroPastoralHousehold	Revenue	None
available_fertilizer_stock	Markets	Available Fertilizer Stock	None

Le processus agit sur les variables suivantes :

Variables d'état	Entité	Définition	Unité
fertilizer_stock	AgroPastoralHousehold	Stock of fertilizer available to the household for use in crop production (forage or cereals).	None
cash	AgroPastoralHousehold	Revenue	None
available_fertilizer_stock	Markets	Available Fertilizer Stock	None
fertilizer_stock	AgroPastoralHousehold	Stock of fertilizer available to the household for use in crop production (forage or cereals).	None

et dépend des paramètres suivants :

Paramètres	Entité	Définition	Unité
fertilizer_price	Parameter	Fertilizer Price	currency/kg
quantity	Parameter	quantity	kg

Comportement

Description détaillée du processus (équations, états, activités, etc.).

Équation :

`Cost = Qty * PriceFert` `ΔStock = Qty` `ΔCash = -Cost`

Pseudo-code

Pseudo-code tel que fourni par le modèle.

```
 // Calculate Cost (from parameters)
cost = quantity * fertilizer_price /* property_023 */;
purchased_qty = quantity /* property_024 */;

// Decision: Check affordability
if (cash >= cost and available_fertilizer_stock >= purchased_qty) {
  // Execute Purchase

  // 1. Decrease AgroPastoralHouseholds Cash
  cash /* resource_014 */ = cash - cost;

  // 2. Decrease Market Stock
  available_fertilizer_stock /* resource_037 */ = available_fertilizer_stock - purchased_qty;

  // 3. Increase AgroPastoralHouseholds Stock
  Fertilizer_Stock /* resource_015 */ = Fertilizer_Stock + purchased_qty;
} else {
  // Purchase failed (not enough cash or market stock)
  purchased_qty = 0;
}
```

Processus: biodiversity_conservation

Documentation :

Manages the state of the Biodiversity resource, recognizing that moderate grazing is beneficial, but excessive pressure causes degradation linked to soil erosion risk.

Attributs

Le processus dépend des variables suivantes :

Variables d'état	Entité	Définition	Unité
biodiversity	Biodiversity	Biodiversity	None
herd_tlu	AgroPastoralHousehold	The aggregated livestock population of the Agro-Pastoral Households, converted to Tropical Livestock Units (TLU).	None
seasonal_fodder	HerbaceousCover	Seasonal Fodder	None
soil_nitrogen	Soil	Nitrogen in the soil, essential for fertility	kg

Le processus agit sur les variables suivantes :

Variables d'état	Entité	Définition	Unité
biodiversity	Biodiversity	Biodiversity	None
soil_nitrogen	Soil	Nitrogen in the soil, essential for fertility	kg

et dépend des paramètres suivants :

Paramètres	Entité	Définition	Unité

Paramètres	Entité	Définition	Unité
biodiversity_growth_rate	Parameter	The maximum natural or assisted rate of recovery for the Biodiversity resource.	1/step
grazing_degradation_actor	Parameter	The sensitivity coefficient determining how much biodiversity is lost per unit of excessive grazing pressure.	1/TLU
sustain_graz_threshold	Parameter	The maximum TLU/Area ratio the ecosystem can sustain before degradation begins (Biodiversity Index decreases).	TLU/ha
soil_erosion_factor	Parameter	The coefficient determining the rate of Soil Nitrogen (resource_001) loss per unit of excessive grazing pressure.	kgN/TLU/step

Comportement

Description détaillée du processus (équations, états, activités, etc.).

Équation :

```
\Delta \text{Biod} = f(\text{GrazingPressure}, \text{Thresholds})
```

Pseudo-code

Pseudo-code tel que fourni par le modèle.

```
 // 1. Define Parameters and Inputs
// (Assuming Total_Grazing_Area is a known parameter or aggregate of Grasslands/Rangelands area)
herd_tlu_tmp = herd_tlu /* resource_069 */;
grazing_pressure = herd_tlu_tmp / total_grazing_area;

degradation_threshold = sustain_graz_threshold /* property_098 */;
optimal_threshold = sustain_graz_threshold /* property_137 */;

// 2. Determine Biodiversity Change (Three Regimes for Realism)
if (grazing_pressure < optimal_threshold) {
  // Regime 1: Under-grazing or Light Grazing (Sub-optimal growth, as grazing is necessary)
  change = biodiversity_growth_rate /* property_135 */;
  soil_loss = 0;
} else if (grazing_pressure <= degradation_threshold) {
  // Regime 2: Optimal to Sustainable Grazing (Growth maximized/maintained)
  change = biodiversity_growth_rate /* property_135 */;
  soil_loss = 0;
} else {
  // Regime 3: Over-grazing (Degradation occurs, leading to erosion risk)
  degradation_intensity = grazing_pressure - degradation_threshold;
  change = degradation_intensity * grazing_degradation_factor /* property_136 */;

  // Secondary effect: Link to Soil Erosion Factor (property_144) and loss of Soil Nitrogen
  soil_loss = degradation_intensity * soil_erosion_factor /* property_144 */;
}

// 3. Update Resources (Mass Balance)
// Update Biodiversity (resource_043 assumed)
biodiversity /* resource_043 */ = biodiversity + change;
biodiversity /* resource_043 */ = max(0.0, min(1.0, biodiversity /* resource_043 */));

// Link degradation to soil nutrient loss
soil_nitrogen /* resource_001 */ = soil_nitrogen - soil_loss;
```

Processus: income_diversification

Documentation :

Consolidation of various revenue streams (livestock, crops, off-farm, nonfarm) to model household resilience and update cash reserves. This process represents the inflow of money from diversified sources.

Attributs

Le processus dépend des variables suivantes :

Variables d'état	Entité	Définition	Unité
cash	AgroPastoralHousehold	Revenue	None
off_farm_income	AgroPastoralHousehold	Cash earned from off-farm or non-pastoral labor activities.	None
livestock_revenue	AgroPastoralHousehold	Total cash generated from the sale of livestock (live animals and milk).	None
crop_sale_revenue	AgroPastoralHousehold	Total cash generated from the sale of crop yield (e.g., cereals, legumes).	None

Le processus agit sur les variables suivantes :

Variables d'état	Entité	Définition	Unité
cash	AgroPastoralHousehold	Revenue	None
off_farm_income	AgroPastoralHousehold	Cash earned from off-farm or non-pastoral labor activities.	None
livestock_revenue	AgroPastoralHousehold	Total cash generated from the sale of livestock (live animals and milk).	None
crop_sale_revenue	AgroPastoralHousehold	Total cash generated from the sale of crop yield (e.g., cereals, legumes).	None

et dépend des paramètres suivants :

Paramètres	Entité	Définition	Unité
...	...	...	...

Comportement

Description détaillée du processus (équations, états, activités, etc.).

Équation :

```
\Delta Cash = \sum (\text{Revenue Streams})
```

Pseudo-code

Pseudo-code tel que fourni par le modèle.

```
 // Step 1: Define Income Streams (Assumed outputs from previous processes)
livestock_revenue_tmp = livestock_revenue /* resource_067 */;
crop_revenue_tmp = crop_sale_revenue /* resource_068 */;
nonfarm_offfarm_income = off_farm_income /* resource_066 */;

current_cash = cash /* resource_014 */;

// Step 2: Calculate Total Income Consolidated (Mass Balance)
total_diversified_income = livestock_revenue_tmp + crop_revenue_tmp + nonfarm_offfarm_income;

// Step 3: Update Resources (Cash Inflow and Resource Reset)
cash /* resource_014 */ = cash + total_diversified_income;

livestock_revenue /* resource_067 */ = livestock_revenue - livestock_revenue_tmp;
crop_sale_revenue /* resource_068 */ = crop_sale_revenue - crop_revenue_tmp;
off_farm_income /* resource_066 */ = off_farm_income - nonfarm_offfarm_income;

// Step 4: Update Resilience Property (Qualitative Outcome - Essential for Realism)
// The Resilience Index increases based on the number of active, non-zero revenue streams.
active_streams = 0;
if (livestock_revenue_tmp > 0) {
  active_streams = active_streams + 1;
}
if (crop_revenue_tmp > 0) {
  active_streams = active_streams + 1;
}
if (nonfarm_offfarm_income > 0) {
  active_streams = active_streams + 1;
}

// Placeholder for the resilience function (to be defined precisely later)
Resilience_Index = Resilience_Index + f(active_streams, relative_magnitude);
// Note: For example, combining Farm + Off-Farm yields a specific, statistically significant improvem
```

Processus: haymaking

Documentation :

Conversion of seasonal fodder into storable hay reserves, considering conservation losses and AgroPastoralHouseholds allocation, constrained by labor.

Attributs

Le processus dépend des variables suivantes :

Variables d'état	Entité	Définition	Unité
seasonal_fodder	HerbaceousCover	Seasonal Fodder	None
hay	AgroPastoralHousehold	Stored, conserved seasonal fodder used as supplementary feed.	None
available_labor	AgroPastoralHousehold	The total labor capacity (e.g., person-hours) available to the Households agent.	person-hours

Le processus agit sur les variables suivantes :

Variables d'état	Entité	Définition	Unité
seasonal_fodder	HerbaceousCover	Seasonal Fodder	None
hay	AgroPastoralHousehold	Stored, conserved seasonal fodder used as supplementary feed.	None
available_labor	AgroPastoralHousehold	The total labor capacity (e.g., person-hours) available to the Households agent.	person-hours

et dépend des paramètres suivants :

Paramètres	Entité	Définition	Unité
hay_conservation_rate	Parameter	The proportional amount of Seasonal Fodder allocated for haymaking and storage (typically run at the end of the season).	proportion
hay_efficiency	Parameter	The proportional mass retained after accounting for harvesting, drying, and storage losses.	proportion
haymaking_labor_req_rate	Parameter	Labor units required per kilogram of fodder allocated for haymaking.	man-day/kgDM

Comportement

Description détaillée du processus (équations, états, activités, etc.).

Équation :

```
HayProduced = FodderAllocated * LaborFactor * HayEfficiency
```

Pseudo-code

Pseudo-code tel que fourni par le modèle.

```
 // 1. Get available fodder and parameters
seasonal_fodder_stock = seasonal_fodder /* resource_041 */;
available_labor_stock = available_labor /* resource_064 */;

target_conservation_rate = hay_conservation_rate /* property_126 */;
hay_efficiency = hay_efficiency /* property_127 */;
labor_needed_rate = haymaking_labor_req_rate /* property_128 */;

// 2. Conditional Execution (Run only if there is stock and intent to conserve)
if (seasonal_fodder_stock > 0 and target_conservation_rate > 0) {
  // Amount the household *wants* to allocate for haymaking
  fodder_allocated_potential = seasonal_fodder_stock * target_conservation_rate;
  labor_needed_potential = fodder_allocated_potential * labor_needed_rate;

  // Determine the labor constraint factor (1 if enough labor, < 1 if labor limited)
  if (labor_needed_potential > 0) {
    labor_constraint_factor = min(1, available_labor_stock / labor_needed_potential);
  } else {
    labor_constraint_factor = 1;
  }

  // Fodder converted is constrained by labor, desire, and stock availability
  fodder_converted_by_labor = fodder_allocated_potential * labor_constraint_factor;

  // Final mass consumed from fodder stock (cannot exceed available stock)
  mass_consumed_from_fodder = min(fodder_converted_by_labor, seasonal_fodder_stock);

  // 3. Calculate final Hay Produced and Consumption
  hay_produced = mass_consumed_from_fodder * hay_efficiency;

  // Actual labor consumed (based on the amount actually converted)
  labor_consumed = mass_consumed_from_fodder * labor_needed_rate;

  // 4. Update Resources (Mass Balance)

  // Consume source (Seasonal Fodder resource_041)
  seasonal_fodder /* resource_041 */ = seasonal_fodder - mass_consumed_from_fodder;

  // Increase destination (Hay resource_032)
  hay /* resource_032 */ = hay + hay_produced;

  // Consume Labor (resource_064)
  available_labor /* resource_064 */ = available_labor - labor_consumed;
} else {
  // Zero out if conditions are not met
  hay_produced = 0;
  mass_consumed_from_fodder = 0;
  labor_consumed = 0;
}
```

Processus: crop_sale

Documentation :

Models the strategic decision of the household to sell a proportion of their crop yield to the market, generating Crop Sale Revenue.

Attributs

Le processus dépend des variables suivantes :

Variables d'état	Entité	Définition	Unité
crop_yield	AgroPastoralHousehold	The mass of harvested crop grain/product.	kg
crop_sale_revenue	AgroPastoralHousehold	Total cash generated from the sale of crop yield (e.g., cereals, legumes).	None

Le processus agit sur les variables suivantes :

Variables d'état	Entité	Définition	Unité
crop_yield	AgroPastoralHousehold	The mass of harvested crop grain/product.	kg
crop_sale_revenue	AgroPastoralHousehold	Total cash generated from the sale of crop yield (e.g., cereals, legumes).	None

et dépend des paramètres suivants :

Paramètres	Entité	Définition	Unité
crop_sale_proportion	Parameter	The fractional decision (0-1) by the Agro-Pastoral Households on how much of the Crop Yield to sell.	proportion
crop_market_price	Parameter	The price of crop yield per unit mass (e.g., currency/kg) on the market.	currency/kg

Comportement

Description détaillée du processus (équations, états, activités, etc.).

Équation :

$$\Delta \text{Revenue} = \text{Yield} \times \text{Proportion} \times \text{Price}$$

Pseudo-code

Pseudo-code tel que fourni par le modèle.

```
 // 1. Determine Household Decision and Market Price
sale_proportion = crop_sale_proportion /* property_130 */;
crop_price = crop_market_price /* property_129 */;
current_yield_stock = crop_yield /* resource_028 */;

// Check if there is anything to sell and a decision to sell (proportion > 0)
if (current_yield_stock > 0 and sale_proportion > 0) {
  // 2. Calculate Transaction Quantities (Robustness Check)
  crop_sold_potential = current_yield_stock * sale_proportion;

  // The amount sold cannot exceed the available stock
  crop_sold_actual = min(crop_sold_potential, current_yield_stock);

  revenue_generated = crop_sold_actual * crop_price;

  // 3. Update Resources (Mass Balance)

  // Increase financial revenue (resource_068)
  crop_sale_revenue /* resource_068 */ = crop_sale_revenue + revenue_generated;

  // Consume physical stock (resource_028)
  crop_yield /* resource_028 */ = crop_yield - crop_sold_actual;
} else {
  crop_sold_actual = 0;
  revenue_generated = 0;
}
```

Processus: milk_partitioning

Documentation :

Distributes the produced milk into three strategic uses: household consumption, market sale, and calf rearing (retained milk).

Attributs

Le processus dépend des variables suivantes :

Variables d'état	Entité	Définition	Unité
milk	AgroPastoralHousehold	Milk	None
milk_for_sale	AgroPastoralHousehold	Stock of milk sold by the household for direct consumption or for dairy	L
milk_for_consumption	AgroPastoralHousehold	Stock of milk retained by the household for direct nutritional use, separate from milk sold or left for calves.	L

Le processus agit sur les variables suivantes :

Variables d'état	Entité	Définition	Unité
milk	AgroPastoralHousehold	Milk	None

Variables d'état	Entité	Définition	Unité
milk_for_sale	AgroPastoralHousehold	Stock of milk sold by the household for direct consumption or for dairy	L
natural_litter	Soil	Natural litter	None

et dépend des paramètres suivants :

Paramètres	Entité	Définition	Unité
live_animal_price	Parameter	The average price received per animal (or per unit of weight) sold to the market/slaughterhouse.	currency/animal
slaughter_prop_young_male	Parameter	Fraction of Young Males sold that are destined for the slaughterhouse.	proportion

Comportement

Description détaillée du processus (équations, états, activités, etc.).

Équation :

$$\Delta \text{Revenue} = \text{MilkSold} \times \text{Price}$$

Pseudo-code

Pseudo-code tel que fourni par le modèle.

```
 // 1. Get Inputs and Strategic Decisions
// Total raw milk produced by AgroPastoralHousehold
total_milk = milk /* resource_039 */;
consumption_prop = milk_consumption_proportion /* property_145 */;
sale_prop = sale_milk_proportion /* property_012 */;

// 2. Calculate Partitioned Quantities
milk_sold = total_milk * sale_prop; /* resource_071 */
milk_consumed = total_milk * consumption_prop; /* autoconsumption */
milk_retained = total_milk - milk_consumed - milk_sold; // Residual

// 3. Update Resources (Mass Balance)
// Consume the total produced milk stock
milk /* resource_039 */ = 0;

// Add Milk for Consumption (assuming a resource for household consumption stock)
milk_for_consumption /* resource_007 */ = milk_for_consumption + milk_consumed;

// Note: The Milk_Retained_Calf will affect the Body Condition/Health of calves
// in a separate process (e.g., Feeding).
```

Entité : Herd (agent_006)

Documentation

Pastoral herd

Attributs

Les ressources détenues par l'agent :

Paramètre	Commentaire	Unité
`young_male_count`	Young Male Count	None
`young_female_count`	Young Female Count	None
`sub_adult_male_count`	Sub-Adult Male Count	None
`sub_adult_female_count`	Sub-Adult Female Count	None
`adult_male_count`	Adult Male Count	None
`adult_female_count`	Adult Female Count	None
`body_condition`	Body Condition (body condition score (BCS))	None
`body_weight`	Body Weight (live weight (LW))	None
`faeces`	Animal manure	kg
`urine`	Animal urine	liters
`livestock_health`	Livestock Health	None
`nutrient_consumed`	Nutrient consumed by herd	mg/kg
`daily_intake_ratio`	The ratio of Actual Fodder Intake to Required Fodder Intake (from process_003). Key driver for Body Condition.	dimensionless
`hydration_stress_factor`	Index (0 to 1) quantifying stress due to water deficit (0=none, 1=severe dehydration). Output of watering process.	dimensionless

Dynamiques

Processus: herd_demography

Documentation :

Herd demographic dynamics: Births, aging, and mortality driven by environmental stress and animal health.

Attributs

Le processus dépend des variables suivantes :

Variables d'état	Entité	Définition	Unité
young_male_count	Herd	Young Male Count	None

Variables d'état	Entité	Définition	Unité
young_female_count	Herd	Young Female Count	None
sub_adult_male_count	Herd	Sub-Adult Male Count	None
sub_adult_female_count	Herd	Sub-Adult Female Count	None
adult_male_count	Herd	Adult Male Count	None
adult_female_count	Herd	Adult Female Count	None
livestock_health	Herd	Livestock Health	None
body_condition	Herd	Body Condition (body condition score (BCS))	None

Le processus agit sur les variables suivantes :

Variables d'état	Entité	Définition	Unité
young_male_count	Herd	Young Male Count	None
young_female_count	Herd	Young Female Count	None
sub_adult_male_count	Herd	Sub-Adult Male Count	None
sub_adult_female_count	Herd	Sub-Adult Female Count	None
adult_male_count	Herd	Adult Male Count	None
adult_female_count	Herd	Adult Female Count	None

et dépend des paramètres suivants :

Paramètres	Entité	Définition	Unité
young_male_mortality_rate	Parameter	Young Male Mortality Rate	proportion/step
young_female_mortality_rate	Parameter	Young Female Mortality Rate	proportion/step
sub_adult_male_mortality_rate	Parameter	Sub-Adult Male Mortality Rate	proportion/step
sub_adult_female_mortality_rate	Parameter	Sub-Adult Female Mortality Rate	proportion/step
adult_male_mortality_rate	Parameter	Adult Male Mortality Rate	proportion/step
adult_female_mortality_rate	Parameter	Adult Female Mortality Rate	proportion/step
sub_adult_age_duration	Parameter	The duration (in years/steps) an animal spends in the sub-adult class before transitioning to the adult class.	years
young_male_min_age	Parameter	Young Male Min Age	years
young_male_max_age	Parameter	Young Male Max Age	years

Paramètres	Entité	Définition	Unité
young_female_min_age	Parameter	Minimum age (in years/steps) for female animals to transition out of the young class.	years
young_female_max_age	Parameter	Maximum age (in years/steps) for female animals to transition out of the young class.	years
adult_female_reprod_rate	Parameter	Adult Female Reproduction Rate	offspring/female/step
sub_adult_female_reprod_rate	Parameter	Sub-Adult Female Reproduction Rate	offspring/female/step
sex_ratio	Parameter	Proportion of male offspring at birth (Male Count / Total Births).	proportion
drought_mortality_rate	Parameter	Drought Mortality Rate	proportion/step

Comportement

Description détaillée du processus (équations, états, activités, etc.).

Équation :

```
ΔN(class) = Births(class) - Mortality(class) + Inflow(class) - Outflow(class)
```

Pseudo-code

Pseudo-code tel que fourni par le modèle.

```
 // 0. Retrieve Parameters and Status
// Reproduction Parameters
adult_female_reprod_rate = adult_female_reprod_rate /* property_016 */;
sub_adult_female_reprod_rate = sub_adult_female_reprod_rate /* property_017 */;
sex_ratio_male = sex_ratio /* property_101 */;

// Mortality and Status
drought_mortality_rate = drought_mortality_rate /* property_036 */;
livestock_health = livestock_health /* resource_040 */; // Scale 0 (worst) to 1 (best)
body_condition = body_condition /* resource_046 */; // Scale 0 (worst) to 1 (best)

// Aging Durations
young_male_duration = young_male_max_age /* property_015 */ - young_male_min_age /* property_014 */;
young_female_duration = young_female_max_age /* property_103 */ - young_female_min_age /* property_10
sub_adult_duration = sub_adult_age_duration /* property_104 */;

// 1. Calculate Overall Mortality Multiplier (Health and Stress Link)
// Health Deficit (1.0 = perfect health, 0.0 = average health/condition is 0)
health_status_score = (livestock_health + body_condition) / 2.0;
health_deficit_factor = max(0.0, 1.0 - health_status_score);

// Use a sensitivity factor (Placeholder for future property, e.g., 0.5)
health_impact_sensitivity = 0.5;

// Overall Multiplier (Base 1.0 + Drought Shock + Health/Condition Penalty)
overall_mortality_multiplier = 1.0 + drought_mortality_rate + (health_deficit_factor * health_impact_
overall_mortality_multiplier = max(1.0, overall_mortality_multiplier);

// 2. Calculate Births (Applies to Young classes)
total_births = (adult_female_count /* resource_025 */ * adult_female_reprod_rate)
```

```
            + (sub_adult_female_count /* resource_023 */ * sub_adult_female_reprod_rate);

young_male_count_inflow = total_births * sex_ratio_male;
young_female_count_inflow = total_births * (1.0 - sex_ratio_male);

// 3. Calculate Mortality and Aging Transitions (Temporal Update)

// A. Young Male Class Transition (YM -> SAM)
aging_out_YM = young_male_count /* resource_019 */ * (1.0 / max(1.0, young_male_duration));
deaths_YM = young_male_count /* resource_019 */ * young_male_mortality_rate /* property_013 */ * over

sub_adult_male_count /* resource_022 */ = sub_adult_male_count + aging_out_YM;
young_male_count /* resource_019 */ = max(0.0, young_male_count + young_male_count_inflow - deaths_YM

// B. Young Female Class Transition (YF -> SAF)
aging_out_YF = young_female_count /* resource_021 */ * (1.0 / max(1.0, young_female_duration));
deaths_YF = young_female_count /* resource_021 */ * young_female_mortality_rate /* property_075 */ *

sub_adult_female_count /* resource_023 */ = sub_adult_female_count + aging_out_YF;
young_female_count /* resource_021 */ = max(0.0, young_female_count + young_female_count_inflow - dea

// C. Sub-Adult Male Class Transition (SAM -> AM)
aging_out_SAM = sub_adult_male_count /* resource_022 */ * (1.0 / max(1.0, sub_adult_duration));
deaths_SAM = sub_adult_male_count /* resource_022 */ * sub_adult_male_mortality_rate /* property_076

adult_male_count /* resource_024 */ = adult_male_count + aging_out_SAM;
sub_adult_male_count /* resource_022 */ = max(0.0, sub_adult_male_count - deaths_SAM - aging_out_SAM)

// D. Sub-Adult Female Class Transition (SAF -> AF)
aging_out_SAF = sub_adult_female_count /* resource_023 */ * (1.0 / max(1.0, sub_adult_duration));
deaths_SAF = sub_adult_female_count /* resource_023 */ * sub_adult_female_mortality_rate /* property_

adult_female_count /* resource_025 */ = adult_female_count + aging_out_SAF;
sub_adult_female_count /* resource_023 */ = max(0.0, sub_adult_female_count - deaths_SAF - aging_out_

// E. Adult Classes (Only Mortality)
deaths_AM = adult_male_count /* resource_024 */ * adult_male_mortality_rate /* property_078 */ * over
adult_male_count /* resource_024 */ = max(0.0, adult_male_count - deaths_AM);

deaths_AF = adult_female_count /* resource_025 */ * adult_female_mortality_rate /* property_079 */ *
adult_female_count /* resource_025 */ = max(0.0, adult_female_count - deaths_AF);

// 4. Track Total Mortality (for output/metrics)
total_mortality = deaths_YM + deaths_YF + deaths_SAM + deaths_SAF + deaths_AM + deaths_AF;
// Suggestion: assigner total_mortality à une ressource dédiée pour suivi, ex. :
// total_mortality_resource /* resource_XXX */ = total_mortality;
```

Processus: manure_production

Documentation :

Manure production by livestock

Attributs

Le processus dépend des variables suivantes :

Variables d'état	Entité	Définition	Unité

Variables d'état	Entité	Définition	Unité
faeces	Herd	Animal manure	kg
urine	Herd	Animal urine	liters

Le processus agit sur les variables suivantes :

Variables d'état	Entité	Définition	Unité
manure	Soil	Manure	kg
faeces	Herd	Animal manure	kg
urine	Herd	Animal urine	liters

et dépend des paramètres suivants :

Paramètres	Entité	Définition	Unité
manure_deposition_rate	Parameter	Fraction of raw excreted waste (faeces + urine) that is effectively deposited and contributes to the Manure resource, accounting for losses like volatilization.	proportion

Comportement

Description détaillée du processus (équations, états, activités, etc.).

Équation :

`ΔManure=(Feces + Urine) * Deposition_Rate`

Pseudo-code

Pseudo-code tel que fourni par le modèle.

```
 // 1. Get raw waste production from the previous time step (outputs of process_023)
total_waste_mass = faeces /* resource_049 */ + urine /* resource_050 */;

// 2. Apply deposition/loss rate
deposition_factor = manure_deposition_rate /* property_105 */;
final_manure_addition = total_waste_mass * deposition_factor;

// 3. Update the Manure Resource
manure /* resource_031 */ = manure + final_manure_addition;

// 4. Clear/Reset the intermediate waste resources
faeces /* resource_049 */ = 0;
urine /* resource_050 */ = 0;
```

Processus: nutrient_absorption

Documentation :

Nutrient absorption, metabolism, and excretion by livestock

Attributs

Le processus dépend des variables suivantes :

Variables d'état	Entité	Définition	Unité
nutrient_consumed	Herd	Nutrient consumed by herd	mg/kg
livestock_health	Herd	Livestock Health	None
body_condition	Herd	Body Condition (body condition score (BCS))	None

Le processus agit sur les variables suivantes :

Variables d'état	Entité	Définition	Unité
faeces	Herd	Animal manure	kg
urine	Herd	Animal urine	liters
nutrient_consumed	Herd	Nutrient consumed by herd	mg/kg

et dépend des paramètres suivants :

Paramètres	Entité	Définition	Unité
base_absorption_rate	Parameter	The base efficiency of nutrient absorption in a healthy, average-condition animal (before health/condition modifiers).	proportion
feces_nutrient_proportion	Parameter	The fraction of non-absorbed (excreted) nutrients that is expelled as feces.	proportion
urine_nutrient_proportion	Parameter	The fraction of non-absorbed (excreted) nutrients that is expelled as urine.	proportion

Comportement

Description détaillée du processus (équations, états, activités, etc.).

Équation :

`NutrientAbsorbed = IntakeN * AbsorptionCapacity` `ExcretedN = IntakeN - NutrientAbsorbed` `WasteOutput = ExcretedN * WasteProportion`

Pseudo-code

Pseudo-code tel que fourni par le modèle.

```
 // Input: Total mass of nutrients consumed in this step (from resource_051, set by process_003)
nutrients_consumed = nutrients_consumed /* resource_051 */;

// 1. Calculate Absorption
// Calculate absorption capacity based on animal factors
absorption_capacity = base_absorption_rate * body_condition * livestock_health /* property_086 resour
absorption_rate = min(1.0, absorption_capacity); // Cap absorption at 100%

// Calculate absorbed and excreted nutrients
absorbed_nutrients = nutrients_consumed * absorption_rate;
excreted_nutrients = nutrients_consumed - absorbed_nutrients;

// 2. Calculate Excretion Components
// Use dedicated parameters for splitting the excreted mass
feces_output = excreted_nutrients * Feces_Nutrient_Proportion /* property_080 */;
urine_output = excreted_nutrients * Urine_Nutrient_Proportion /* property_082 */;

// 3. Update Resources and Status
// Update internal animal state
update_nutrient_levels(absorbed_nutrients);
update_metabolism_status();

// Accumulate waste products (outputs for process_011)
faeces /* resource_049 */ = faeces + feces_output;
urine /* resource_050 */ = urine + urine_output;

// Consume the input resource (Mass Balance: it has been fully processed)
nutrients_consumed /* resource_051 */ = 0;
```

Entité : Grasslands (agent_008)

Documentation

Grasslands for grazing

Attributs

Les ressources détenues par l'agent :

Paramètre	Commentaire	Unité
...	...	...

Dynamiques

Section vide : à compléter manuellement

Entité : WaterSource (agent_012)

Documentation

Water points (wells, ponds) for livestock watering

Attributs

Les ressources détenues par l'agent :

Paramètre	Commentaire	Unité
`water`	Water	None

Dynamiques

Section vide : à compléter manuellement

Entité : Markets (agent_013)

Documentation

Markets for trading pastoral products

Attributs

Les ressources détenues par l'agent :

Paramètre	Commentaire	Unité
`available_fertilizer_stock`	Available Fertilizer Stock	None
`available_concentrate_stock`	Available Concentrate Stock	None

Dynamiques

Section vide : à compléter manuellement

Entité : Slaughterhouse (agent_022)

Documentation

Facilities for processing livestock

Attributs

Les ressources détenues par l'agent :

Paramètre	Commentaire	Unité
`ym_to_slaughter`	Count of Young Male animals transferred to the slaughterhouse	Count

Paramètre	Commentaire	Unité
`yf_to_slaughter`	Count of Young Female animals transferred to the slaughterhouse	Count
`sam_to_slaughter`	Count of Sub-Adult Male animals transferred to the slaughterhouse	Count
`saf_to_slaughter`	Count of Sub-Adult Female animals transferred to the slaughterhouse	Count
`am_to_slaughter`	Count of Adult Male animals transferred to the slaughterhouse	Count
`af_to_slaughter`	Count of Adult Female animals transferred to the slaughterhouse	Count
`carcass`	Carcass	None
`meat_production`	Total meat produced from slaughtered animals	None
`meat_sold_amount`	The total mass (kg) of processed meat sold to the market over the simulation period.	kg

Dynamiques

Processus: slaughtering

Documentation :

Animal slaughter and processing into carcass based on received animals.

Attributs

Le processus dépend des variables suivantes :

Variables d'état	Entité	Définition	Unité
ym_to_slaughter	Slaughterhouse	Count of Young Male animals transferred to the slaughterhouse	Count
yf_to_slaughter	Slaughterhouse	Count of Young Female animals transferred to the slaughterhouse	Count
sam_to_slaughter	Slaughterhouse	Count of Sub-Adult Male animals transferred to the slaughterhouse	Count
saf_to_slaughter	Slaughterhouse	Count of Sub-Adult Female animals transferred to the slaughterhouse	Count
am_to_slaughter	Slaughterhouse	Count of Adult Male animals transferred to the slaughterhouse	Count
af_to_slaughter	Slaughterhouse	Count of Adult Female animals transferred to the slaughterhouse	Count
body_condition	Herd	Body Condition (body condition score (BCS))	None
body_weight	Herd	Body Weight (live weight (LW))	None

Le processus agit sur les variables suivantes :

Variables d'état	Entité	Définition	Unité
carcass	Slaughterhouse	Carcass	None
meat_production	Slaughterhouse	Total meat produced from slaughtered animals	None

Variables d'état	Entité	Définition	Unité
ym_to_slaughter	Slaughterhouse	Count of Young Male animals transferred to the slaughterhouse	Count
yf_to_slaughter	Slaughterhouse	Count of Young Female animals transferred to the slaughterhouse	Count
sam_to_slaughter	Slaughterhouse	Count of Sub-Adult Male animals transferred to the slaughterhouse	Count
saf_to_slaughter	Slaughterhouse	Count of Sub-Adult Female animals transferred to the slaughterhouse	Count
am_to_slaughter	Slaughterhouse	Count of Adult Male animals transferred to the slaughterhouse	Count
af_to_slaughter	Slaughterhouse	Count of Adult Female animals transferred to the slaughterhouse	Count

et dépend des paramètres suivants :

Paramètres	Entité	Définition	Unité
body_weight	Parameter	Body Weight	kg/animal
base_dressing_percentage	Parameter	The base percentage of live animal weight that converts to carcass weight under standard/optimal conditions.	proportion
optimal_condition	Parameter	The Body Condition Score (BCS) value representing optimal health and body reserves, used as a reference for carcass yield and stress calculation.	BCS

Comportement

Description détaillée du processus (équations, états, activités, etc.).

Équation :

```
CarcassWeight = NumToSlaughter * AvgLiveWeight * BaseDressPct * ConditionFactor
```

Pseudo-code

Pseudo-code tel que fourni par le modèle.

```
 // Initialize total carcass production
total_carcass = 0;

// Get condition factor (common to all classes)
condition_factor = body_condition / Optimal_Condition /* resource_046 property_114 */;
base_dressing = base_dressing_percentage /* property_098 */;

// Calculate carcass yield per class (explicit per category, no loop)
ym_carcass = ym_to_slaughter * weight_ym * base_dressing * condition_factor /* resource_054 */;
yf_carcass = yf_to_slaughter * weight_yf * base_dressing * condition_factor /* resource_055 */;
sam_carcass = sam_to_slaughter * weight_sam * base_dressing * condition_factor /* resource_056 */;
saf_carcass = saf_to_slaughter * weight_saf * base_dressing * condition_factor /* resource_057 */;
am_carcass = am_to_slaughter * weight_am * base_dressing * condition_factor /* resource_058 */;
af_carcass = af_to_slaughter * weight_af * base_dressing * condition_factor /* resource_059 */;

// Sum all class contributions
total_carcass = ym_carcass + yf_carcass + sam_carcass + saf_carcass + am_carcass + af_carcass;

// Reset input flow resources (all consumed by slaughter)
ym_to_slaughter = 0; /* resource_054 */
yf_to_slaughter = 0; /* resource_055 */
sam_to_slaughter = 0; /* resource_056 */
saf_to_slaughter = 0; /* resource_057 */
am_to_slaughter = 0; /* resource_058 */
af_to_slaughter = 0; /* resource_059 */

// Update production resources (Mass Balance)
Carcass /* resource_017 */ = Carcass + total_carcass;
meat_production /* resource_036 */ = meat_production + total_carcass;
```

Processus: meat_sale

Documentation :

Sale of processed meat stock, tracking the cumulative volume sold.

Attributs

Le processus dépend des variables suivantes :

Variables d'état	Entité	Définition	Unité
am_to_slaughter	Slaughterhouse	Count of Adult Male animals transferred to the slaughterhouse	Count
meat_sold_amount	Slaughterhouse	The total mass (kg) of processed meat sold to the market over the simulation period.	kg

Le processus agit sur les variables suivantes :

Variables d'état	Entité	Définition	Unité
am_to_slaughter	Slaughterhouse	Count of Adult Male animals transferred to the slaughterhouse	Count
meat_sold_amount	Slaughterhouse	The total mass (kg) of processed meat sold to the market over the simulation period.	kg

et dépend des paramètres suivants :

Paramètres	Entité	Définition	Unité
market_price_meat	Parameter	Market Price for meat after all processing in slaughterhouse	currency/kg

Comportement

Description détaillée du processus (équations, états, activités, etc.).

Équation :

`MeatSoldCum += ProcessedMeatStock` `DeltaStock = -ProcessedMeatStock`

Pseudo-code

Pseudo-code tel que fourni par le modèle.

```
 // quantity to sell is the current stock
to_sell = processed_meat_stock /* resource_058 */;
market_price = market_price_meat /* property_030 */;

if (to_sell > 0) {
  // Calculate revenue (The financial transaction is abstracted away, but revenue is calculated for c
  revenue = to_sell * market_price;

  // Execute transaction: Decrease Stock and Update Cumulative Metric

  // 1. Decrease Stock (consumed)
  processed_meat_stock = processed_meat_stock - to_sell;  /* resource_058 */

  // 2. Update Cumulative Meat Sold Amount
  Meat_Sold_Amount_Cumulative = Meat_Sold_Amount_Cumulative + to_sell /* resource_061 */;
 }
else {
  // No stock to sell, no change
  revenue = 0;
  }
```

Processus: meat_processing

Documentation :

Processing of carcasses into meat products

Attributs

Le processus dépend des variables suivantes :

Variables d'état	Entité	Définition	Unité
carcass	Slaughterhouse	Carcass	None

Le processus agit sur les variables suivantes :

Variables d'état	Entité	Définition	Unité

Variables d'état	Entité	Définition	Unité
processed_meat_stock	Slaughterhouse	The final inventory of packaged, salable meat product ready for market.	kg
carcass	Slaughterhouse	Carcass	None

et dépend des paramètres suivants :

Paramètres	Entité	Définition	Unité
trimming_efficiency	Parameter	The proportional weight retained after removing external fat/waste during trimming (0 to 1).	proportion
cutting_efficiency	Parameter	The proportional weight retained after bone/excess cutting waste (0 to 1).	proportion
packaging_loss	Parameter	The proportional weight lost during packaging due to moisture/handling (0 to 1).	proportion
storage_loss	Parameter	The proportional weight lost during cold storage (0 to 1).	proportion

Comportement

Description détaillée du processus (équations, états, activités, etc.).

Équation :

```
ProcessedMeat = Carcass * TrimmingEff * CuttingEff * (1 - PackagingLoss) * (1 - StorageLoss)
```

Pseudo-code

Pseudo-code tel que fourni par le modèle.

```
 // Input: Total carcass mass available for processing (resource_017)
carcass_initial_weight = carcass /* resource_017 */;

// If there is carcass to process
if (initial_weight > 0) {
  // 1. Apply trimming efficiency loss
  trimming_efficiency = trimming_efficiency /* property_092 */;
  trimmed_weight = initial_weight * trimming_efficiency;

  // 2. Apply cutting efficiency loss
  cutting_efficiency = cutting_efficiency /* property_093 */;
  cut_weight = trimmed_weight * cutting_efficiency;

  // 3. Apply packaging loss
  packaging_loss = packaging_loss /* property_094 */;
  packaged_weight = cut_weight * (1 - packaging_loss);

  // 4. Apply storage loss
  storage_loss = storage_loss /* property_095 */;
  stored_weight = packaged_weight * (1 - storage_loss);

  // 5. Update Inventory and Production

  // Decrease Carcass stock (consumed input)
  carcass /* resource_017 */ = carcass - carcass_initial_weight;

  // Increase Processed Meat Stock (new salable output)
  Processed_Meat_Stock /* resource_060 */ = Processed_Meat_Stock + stored_weight;

  // Record Waste (difference between input and final output)
  waste_mass = initial_weight - stored_weight;
  // record_waste_statistics(waste_mass)  // placeholder for statistics logging
} else {
  // No carcass to process, no change
  stored_weight = 0;
}
```

Entité : Dairy (agent_023)

Documentation

Agent responsible for processing raw milk into finished dairy products

Attributs

Les ressources détenues par l'agent :

Paramètre	Commentaire	Unité
`dairy_milk`	Raw milk transferred from the AgroPastoralHouseholds for dairy processing.	Liters
`dairy_product_stock`	Processed milk products (e.g., cheese, yogurt) inventory held by the Dairy.	kg or Liters

Paramètre	Commentaire	Unité
`dairy_product_sold_amount`	The total mass or volume of processed dairy products sold over the simulation period.	kg or Liters

Dynamiques

Processus: milk_processing_dairy

Documentation :

Conversion of raw milk input into final dairy products.

Attributs

Le processus dépend des variables suivantes :

Variables d'état	Entité	Définition	Unité
dairy_milk	Dairy	Raw milk transferred from the AgroPastoralHouseholds for dairy processing.	Liters

Le processus agit sur les variables suivantes :

Variables d'état	Entité	Définition	Unité
dairy_milk	Dairy	Raw milk transferred from the AgroPastoralHouseholds for dairy processing.	Liters
dairy_product_stock	Dairy	Processed milk products (e.g., cheese, yogurt) inventory held by the Dairy.	kg or Liters

et dépend des paramètres suivants :

Paramètres	Entité	Définition	Unité
dairy_processing_efficiency	Parameter	The proportional conversion rate of raw milk volume to final processed product mass/volume (0 to 1), accounting for losses and conversion ratio.	proportion

Comportement

Description détaillée du processus (équations, états, activités, etc.).

Équation :

```
ProductOutput = MilkForProcessing * ProcessingEfficiency
```

Pseudo-code

Pseudo-code tel que fourni par le modèle.

```
 // Get raw milk input
raw_milk_input = milk_for_processing /* resource_029 */;
processing_efficiency = dairy_processing_efficiency /* property_117 */;

if raw_milk_input > 0:
  // Calculate final product output
  product_output = raw_milk_input * processing_efficiency;

  // 1. Consume raw milk input
  milk_for_processing = milk_for_processing - raw_milk_input; /* resource_029 */

  // 2. Increase final product stock
  Dairy_Product_Stock = Dairy_Product_Stock + product_output; /* resource_026 */
else:
  product_output = 0;
```

Processus: dairy_product_sale

Documentation :

Sale of processed dairy products at market, tracking cumulative volume sold.

Attributs

Le processus dépend des variables suivantes :

Variables d'état	Entité	Définition	Unité
dairy_product_stock	Dairy	Processed milk products (e.g., cheese, yogurt) inventory held by the Dairy.	kg or Liters
dairy_product_sold_amount	Dairy	The total mass or volume of processed dairy products sold over the simulation period.	kg or Liters

Le processus agit sur les variables suivantes :

Variables d'état	Entité	Définition	Unité
dairy_product_stock	Dairy	Processed milk products (e.g., cheese, yogurt) inventory held by the Dairy.	kg or Liters
dairy_product_sold_amount	Dairy	The total mass or volume of processed dairy products sold over the simulation period.	kg or Liters

et dépend des paramètres suivants :

Paramètres	Entité	Définition	Unité
market_price_dairy_product	Parameter	The market price per unit (kg or L) of the final processed dairy product stock.	currency/kg

Comportement

Description détaillée du processus (équations, états, activités, etc.).

Équation :

`DairySoldCum += DairyProductStock` `DeltaStock = -DairyProductStock`

Pseudo-code

Pseudo-code tel que fourni par le modèle.

```
 // Quantity to sell is the current stock
to_sell = processed_meat_stock /* resource_058 */;
market_price = market_price_meat /* property_030 */;

// Decision: Sell if there is stock available
if (to_sell > 0) {
  // Calculate revenue (The financial transaction is abstracted away, but revenue is calculated for c
  revenue = to_sell * market_price;

  // Execute transaction: Decrease Stock and Update Cumulative Metric

  // 1. Decrease Stock (consumed/sold)
  processed_meat_stock /* resource_058 */ = processed_meat_stock - to_sell;

  // 2. Update Cumulative Meat Sold Amount
  Meat_Sold_Amount_Cumulative /* resource_061 */ = Meat_Sold_Amount_Cumulative + to_sell;
} else {
  // No stock to sell, no change
  revenue = 0;
}
```

Entité : GovernmentalInstitutions (agent_014)

Documentation

Institutions for policies and conflict resolution

Attributs

Les ressources détenues par l'agent :

Paramètre	Commentaire	Unité
...	...	...

Dynamiques

Processus: conflict_resolution

Documentation :

Sequential management of resource conflicts (Water, Fodder) beginning with informal, knowledge-based mediation before involving formal institutions. The successful outcome restores full access to the disputed resources by setting the Conflict Penalty Factor to zero.

Attributs

Le processus dépend des variables suivantes :

Variables d'état	Entité	Définition	Unité
water	WaterSource	Water	None
seasonal_fodder	HerbaceousCover	Seasonal Fodder	None
traditional_knowledge	AgroPastoralHousehold	Traditional Knowledge	None

Le processus agit sur les variables suivantes :

Variables d'état	Entité	Définition	Unité
water	WaterSource	Water	None
seasonal_fodder	HerbaceousCover	Seasonal Fodder	None

et dépend des paramètres suivants :

Paramètres	Entité	Définition	Unité
governance_efficiency	Parameter	The effectiveness (0-1) of Governmental Institutions (agent_014) in resolving resource conflicts.	index
informal_mediation_factor	Parameter	The intrinsic effectiveness (0-1) of local, informal mechanisms (social norms) for conflict resolution, based on Traditional Knowledge.	index
conflict_penalty_water	Parameter	Penalty applied to water availability during active conflict (0 to 1).	proportion
conflict_penalty_seasonal_fodder	Parameter	Penalty applied to seasonal fodder availability during active conflict (0 to 1).	proportion

Comportement

Description détaillée du processus (équations, états, activités, etc.).

Équation :

$$\text{ProbResolution} = f(\text{TraditionalKnowledge}, \text{GovernanceEfficiency})$$

Pseudo-code

Pseudo-code tel que fourni par le modèle.

```
if (conflict_detected) {
  // 1. Tentative de résolution informelle
  ProbInformal = traditional_knowledge /* resource_044 */ * informal_mediation_factor /* property_134

  if (random < ProbInformal) {
    conflict_resolved = true;
  } else {
    // 2. Escalade vers résolution formelle
    ProbFormal = governance_efficiency /* property_133 */;

    if (random < ProbFormal) {
      conflict_resolved = true;
    } else {
      conflict_resolved = false;
    }
  }

  // 3. Application du résultat sur les ressources
  if (conflict_resolved) {
    conflict_penalty_water = 0.0 /* property_164 */;
    conflict_penalty_seasonal_fodder = 0.0 /* property_165 */;
  } else {
    conflict_penalty_water = 0.5;
    conflict_penalty_seasonal_fodder = 0.5;
  }
}
// Fin du bloc de conflit
```

Entité : Biodiversity (agent_015)

Documentation

Wildlife interacting with pastoral ecosystems

Attributs

Les ressources détenues par l'agent :

Paramètre	Commentaire	Unité
`biodiversity`	Biodiversity	None

Dynamiques

Section vide : à compléter manuellement

Entité : NonGovernmentalOrganizations (agent_017)

Documentation

NGOs involved in pastoral ecosystem management

Attributs

Les ressources détenues par l'agent :

Paramètre	Commentaire	Unité
...	...	...

Dynamiques

Processus: biodiversity_conservation

Documentation :

Manages the state of the Biodiversity resource, recognizing that moderate grazing is beneficial, but excessive pressure causes degradation linked to soil erosion risk.

Attributs

Le processus dépend des variables suivantes :

Variables d'état	Entité	Définition	Unité
biodiversity	Biodiversity	Biodiversity	None
herd_tlu	AgroPastoralHousehold	The aggregated livestock population of the Agro-Pastoral Households, converted to Tropical Livestock Units (TLU).	None
seasonal_fodder	HerbaceousCover	Seasonal Fodder	None
soil_nitrogen	Soil	Nitrogen in the soil, essential for fertility	kg

Le processus agit sur les variables suivantes :

Variables d'état	Entité	Définition	Unité
biodiversity	Biodiversity	Biodiversity	None
soil_nitrogen	Soil	Nitrogen in the soil, essential for fertility	kg

et dépend des paramètres suivants :

Paramètres	Entité	Définition	Unité
biodiversity_growth_rate	Parameter	The maximum natural or assisted rate of recovery for the Biodiversity resource.	1/step
grazing_degradation_actor	Parameter	The sensitivity coefficient determining how much biodiversity is lost per unit of excessive grazing pressure.	1/TLU

Paramètres	Entité	Définition	Unité
sustain_graz_threshold	Parameter	The maximum TLU/Area ratio the ecosystem can sustain before degradation begins (Biodiversity Index decreases).	TLU/ha
soil_erosion_factor	Parameter	The coefficient determining the rate of Soil Nitrogen (resource_001) loss per unit of excessive grazing pressure.	kgN/TLU/step

Comportement

Description détaillée du processus (équations, états, activités, etc.).

Équation :

$$\Delta \text{Biod} = f(\text{GrazingPressure}, \text{Thresholds})$$

Pseudo-code

Pseudo-code tel que fourni par le modèle.

```
 // 1. Define Parameters and Inputs
// (Assuming Total_Grazing_Area is a known parameter or aggregate of Grasslands/Rangelands area)
herd_tlu_tmp = herd_tlu /* resource_069 */;
grazing_pressure = herd_tlu_tmp / total_grazing_area;

degradation_threshold = sustain_graz_threshold /* property_098 */;
optimal_threshold = sustain_graz_threshold /* property_137 */;

// 2. Determine Biodiversity Change (Three Regimes for Realism)
if (grazing_pressure < optimal_threshold) {
  // Regime 1: Under-grazing or Light Grazing (Sub-optimal growth, as grazing is necessary)
  change = biodiversity_growth_rate /* property_135 */;
  soil_loss = 0;
} else if (grazing_pressure <= degradation_threshold) {
  // Regime 2: Optimal to Sustainable Grazing (Growth maximized/maintained)
  change = biodiversity_growth_rate /* property_135 */;
  soil_loss = 0;
} else {
  // Regime 3: Over-grazing (Degradation occurs, leading to erosion risk)
  degradation_intensity = grazing_pressure - degradation_threshold;
  change = degradation_intensity * grazing_degradation_factor /* property_136 */;

  // Secondary effect: Link to Soil Erosion Factor (property_144) and loss of Soil Nitrogen
  soil_loss = degradation_intensity * soil_erosion_factor /* property_144 */;
}

// 3. Update Resources (Mass Balance)
// Update Biodiversity (resource_043 assumed)
biodiversity /* resource_043 */ = biodiversity + change;
biodiversity /* resource_043 */ = max(0.0, min(1.0, biodiversity /* resource_043 */));

// Link degradation to soil nutrient loss
soil_nitrogen /* resource_001 */ = soil_nitrogen - soil_loss;
```

Entité : Researchers (agent_018)

Documentation

Researchers studying pastoral ecosystems

Attributs

Les ressources détenues par l'agent :

Paramètre	Commentaire	Unité
...	...	...

Dynamiques

Processus: biodiversity_conservation

Documentation :

Manages the state of the Biodiversity resource, recognizing that moderate grazing is beneficial, but excessive pressure causes degradation linked to soil erosion risk.

Attributs

Le processus dépend des variables suivantes :

Variables d'état	Entité	Définition	Unité
biodiversity	Biodiversity	Biodiversity	None
herd_tlu	AgroPastoralHousehold	The aggregated livestock population of the Agro-Pastoral Households, converted to Tropical Livestock Units (TLU).	None
seasonal_fodder	HerbaceousCover	Seasonal Fodder	None
soil_nitrogen	Soil	Nitrogen in the soil, essential for fertility	kg

Le processus agit sur les variables suivantes :

Variables d'état	Entité	Définition	Unité
biodiversity	Biodiversity	Biodiversity	None
soil_nitrogen	Soil	Nitrogen in the soil, essential for fertility	kg

et dépend des paramètres suivants :

Paramètres	Entité	Définition	Unité
biodiversity_growth_rate	Parameter	The maximum natural or assisted rate of recovery for the Biodiversity resource.	1/step
grazing_degradation_actor	Parameter	The sensitivity coefficient determining how much biodiversity is lost per unit of excessive grazing pressure.	1/TLU

Paramètres	Entité	Définition	Unité
sustain_graz_threshold	Parameter	The maximum TLU/Area ratio the ecosystem can sustain before degradation begins (Biodiversity Index decreases).	TLU/ha
soil_erosion_factor	Parameter	The coefficient determining the rate of Soil Nitrogen (resource_001) loss per unit of excessive grazing pressure.	kgN/TLU/step

Comportement

Description détaillée du processus (équations, états, activités, etc.).

Équation :

$$\Delta \text{Biod} = f(\text{GrazingPressure}, \text{Thresholds})$$

Pseudo-code

Pseudo-code tel que fourni par le modèle.

```
 // 1. Define Parameters and Inputs
// (Assuming Total_Grazing_Area is a known parameter or aggregate of Grasslands/Rangelands area)
herd_tlu_tmp = herd_tlu /* resource_069 */;
grazing_pressure = herd_tlu_tmp / total_grazing_area;

degradation_threshold = sustain_graz_threshold /* property_098 */;
optimal_threshold = sustain_graz_threshold /* property_137 */;

// 2. Determine Biodiversity Change (Three Regimes for Realism)
if (grazing_pressure < optimal_threshold) {
  // Regime 1: Under-grazing or Light Grazing (Sub-optimal growth, as grazing is necessary)
  change = biodiversity_growth_rate /* property_135 */;
  soil_loss = 0;
} else if (grazing_pressure <= degradation_threshold) {
  // Regime 2: Optimal to Sustainable Grazing (Growth maximized/maintained)
  change = biodiversity_growth_rate /* property_135 */;
  soil_loss = 0;
} else {
  // Regime 3: Over-grazing (Degradation occurs, leading to erosion risk)
  degradation_intensity = grazing_pressure - degradation_threshold;
  change = degradation_intensity * grazing_degradation_factor /* property_136 */;

  // Secondary effect: Link to Soil Erosion Factor (property_144) and loss of Soil Nitrogen
  soil_loss = degradation_intensity * soil_erosion_factor /* property_144 */;
}

// 3. Update Resources (Mass Balance)
// Update Biodiversity (resource_043 assumed)
biodiversity /* resource_043 */ = biodiversity + change;
biodiversity /* resource_043 */ = max(0.0, min(1.0, biodiversity /* resource_043 */));

// Link degradation to soil nutrient loss
soil_nitrogen /* resource_001 */ = soil_nitrogen - soil_loss;
```

Entité : Pastoralists (agent_019)

Documentation

Communities practicing pastoralism

Attributs

Les ressources détenues par l'agent :

Paramètre	Commentaire	Unité
...	...	...

Dynamiques

Processus: income_diversification

Documentation :

Consolidation of various revenue streams (livestock, crops, off-farm, nonfarm) to model household resilience and update cash reserves. This process represents the inflow of money from diversified sources.

Attributs

Le processus dépend des variables suivantes :

Variables d'état	Entité	Définition	Unité
cash	AgroPastoralHousehold	Revenue	None
off_farm_income	AgroPastoralHousehold	Cash earned from off-farm or non-pastoral labor activities.	None
livestock_revenue	AgroPastoralHousehold	Total cash generated from the sale of livestock (live animals and milk).	None
crop_sale_revenue	AgroPastoralHousehold	Total cash generated from the sale of crop yield (e.g., cereals, legumes).	None

Le processus agit sur les variables suivantes :

Variables d'état	Entité	Définition	Unité
cash	AgroPastoralHousehold	Revenue	None
off_farm_income	AgroPastoralHousehold	Cash earned from off-farm or non-pastoral labor activities.	None
livestock_revenue	AgroPastoralHousehold	Total cash generated from the sale of livestock (live animals and milk).	None
crop_sale_revenue	AgroPastoralHousehold	Total cash generated from the sale of crop yield (e.g., cereals, legumes).	None

et dépend des paramètres suivants :

Paramètres	Entité	Définition	Unité
...	...	...	...

Comportement

Description détaillée du processus (équations, états, activités, etc.).

Équation :

```
\Delta Cash = \sum (\text{Revenue Streams})
```

Pseudo-code

Pseudo-code tel que fourni par le modèle.

```
 // Step 1: Define Income Streams (Assumed outputs from previous processes)
livestock_revenue_tmp = livestock_revenue /* resource_067 */;
crop_revenue_tmp = crop_sale_revenue /* resource_068 */;
nonfarm_offfarm_income = off_farm_income /* resource_066 */;

current_cash = cash /* resource_014 */;

// Step 2: Calculate Total Income Consolidated (Mass Balance)
total_diversified_income = livestock_revenue_tmp + crop_revenue_tmp + nonfarm_offfarm_income;

// Step 3: Update Resources (Cash Inflow and Resource Reset)
cash /* resource_014 */ = cash + total_diversified_income;

livestock_revenue /* resource_067 */ = livestock_revenue - livestock_revenue_tmp;
crop_sale_revenue /* resource_068 */ = crop_sale_revenue - crop_revenue_tmp;
off_farm_income /* resource_066 */ = off_farm_income - nonfarm_offfarm_income;

// Step 4: Update Resilience Property (Qualitative Outcome - Essential for Realism)
// The Resilience Index increases based on the number of active, non-zero revenue streams.
active_streams = 0;
if (livestock_revenue_tmp > 0) {
  active_streams = active_streams + 1;
}
if (crop_revenue_tmp > 0) {
  active_streams = active_streams + 1;
}
if (nonfarm_offfarm_income > 0) {
  active_streams = active_streams + 1;
}

// Placeholder for the resilience function (to be defined precisely later)
Resilience_Index = Resilience_Index + f(active_streams, relative_magnitude);
// Note: For example, combining Farm + Off-Farm yields a specific, statistically significant improvem
```

Entité : ClimateSystem (agent_020)

Documentation

Climate system affecting pastoral ecosystems

Attributs

Les ressources détenues par l'agent :

Paramètre	Commentaire	Unité
`rainfall`	Rainfall	mm/year
`atmospheric_nitrogen`	Atmospheric nitrogen	kg
`humidity`	Humidity	%
`wind_speed`	Wind Speed	km/h

Dynamiques

Section vide : à compléter manuellement

Entité : PastoralEcosystem (agent_024)

Documentation

Pastoral ecosystem as a whole

Attributs

Les ressources détenues par l'agent :

Paramètre	Commentaire	Unité
`disease_prevalence_index`	An index (0 to 1) reflecting the environmental or epidemic disease pressure on the herd.	dimensionless

Dynamiques

Section vide : à compléter manuellement

Entité : PastoralEcosystemServices (agent_025)

Documentation

Ecosystem services provided by pastoral ecosystems

Attributs

Les ressources détenues par l'agent :

Paramètre	Commentaire	Unité

Paramètre	Commentaire	Unité
`cash`	Revenue	None
`meat_sold_amount`	The total mass (kg) of processed meat sold to the market over the simulation period.	kg
`dairy_product_sold_amount`	The total mass or volume of processed dairy products sold over the simulation period.	kg or Liters

Dynamiques

Section vide : à compléter manuellement

Entité : PastoralSocioEcosystem (agent_026)

Documentation

Socio-ecological system of pastoralism

Attributs

Les ressources détenues par l'agent :

Paramètre	Commentaire	Unité
...	...	...

Dynamiques

Section vide : à compléter manuellement

Entité : PastoralSocioEconomicSystem (agent_027)

Documentation

Socio-economic system of pastoralism

Attributs

Les ressources détenues par l'agent :

Paramètre	Commentaire	Unité
...	...	...

Dynamiques

Section vide : à compléter manuellement

Entité : SocioEcosystem (agent_028)

Documentation

Overall socio-ecological system of pastoralism

Attributs

Les ressources détenues par l'agent :

Paramètre	Commentaire	Unité
...	...	...

Dynamiques

Section vide : à compléter manuellement

Indicateurs

Pastoralisme

Indicateur PA.01.01 : Charge animale instantanée (UBT/ha)

Description

Nombre d'unités de bétail (UBT) par hectare de pâturage disponible. Indicateur clé de pression sur les ressources fourragères.

Calcul

```
CA = (Nombre UBT) / (Surface pâturable)
```

Attributs

L'indicateur dépend des variables suivantes :

variables	Entité	Description	Unité
nombre_ubt	Troupeau	Effectif converti en unité de bétail tropical	UBT
surface_paturage	Espace	Surface accessible au pâturage	ha

Et dépend des paramètres suivants

Paramètres	Entité	Description	Unité	Valeur
facteur_saison	Système	Coefficient saisonnier d'ajustement	coefficient	1.2

Indicateur PA.02.01 : Taux de rotation des pâturages (rotations/an)

Description

Fréquence de rotation des troupeaux entre parcelles. Critique pour la durabilité des parcours.

Calcul

```
TR = (Nombre rotations) / (Période analysée)
```

Attributs

L'indicateur dépend des variables suivantes :

variables	Entité	Description	Unité
nombre_rotations	Troupeau	Nombre de déplacements entre parcelles	n
periode_analyse	Système	Période d'observation	jours

Et dépend des paramètres suivants

Paramètres	Entité	Description	Unité	Valeur
duree_min_sejour	Parcelle	Durée minimale entre deux rotations	jours	21

Indicateur PA.03.01 : Taux de charge soutenable (UBT/ha/an)

Description

Capacité de charge maximale sans dégradation à long terme. Basé sur la productivité fourragère.

Calcul

```
TCS = (Production fourragère annuelle) / (Consommation UBT)
```

Attributs

L'indicateur dépend des variables suivantes :

variables	Entité	Description	Unité
production_fourrage	Parcelle	Biomasse fourragère produite annuellement	kg MS/ha

variables	Entité	Description	Unité
consommation_ubt	Troupeau	Besoin annuel moyen par UBT	kg MS/UBT

Et dépend des paramètres suivants

Paramètres	Entité	Description	Unité	Valeur
facteur_resilience	Ecosystème	Coefficient de résilience des parcours	coefficient	0.85

Indicateur PA.04.01 : Dépendance aux aliments externes (%)

Description

Proportion des besoins alimentaires du bétail couverte par des achats externes. Indicateur de vulnérabilité économique.

Calcul

```
DAE = (Quantité aliments achetés) / (Besoins totaux) * 100
```

Attributs

L'indicateur dépend des variables suivantes :

variables	Entité	Description	Unité
aliments_achetes	Troupeau	Quantité d'aliments complémentaires acquis	kg MS
besoins_totaux	Troupeau	Requête alimentaire annuelle du troupeau	kg MS

Et dépend des paramètres suivants

Paramètres	Entité	Description	Unité	Valeur
prix_moyen_aliment	Marché	Coût moyen des aliments achetés	€/kg MS	0.35

Indicateur PA.05.01 : Productivité laitière spécifique (L/femelle/an)

Description

Production laitière rapportée au nombre de femelles en lactation. Indicateur clé de performance zootechnique.

Calcul

```
PLS = (Production laitière annuelle) / (Nombre femelles)
```

Attributs

L'indicateur dépend des variables suivantes :

variables	Entité	Description	Unité
production_laitiere	Troupeau	Volume total de lait produit sur l'année	L
nombre_femelles	Troupeau	Effectif moyen des femelles en lactation	têtes

Et dépend des paramètres suivants

Paramètres	Entité	Description	Unité	Valeur
duree_lactation	Troupeau	Période standard de lactation	jours	305

Indicateur PA.06.01 : Indice de mobilité pastorale (km/jour)

Description

Distance annuelle moyenne parcourue par le troupeau. Clé pour l'accès aux ressources.

Calcul

```
IMP = (Distance totale) / (Nombre jours)
```

Attributs

L'indicateur dépend des variables suivantes :

variables	Entité	Description	Unité
distance_parcourue	Troupeau	Distance totale annuelle de transhumance	km
jours_paturage	Troupeau	Nombre de jours de pâturage effectifs	jours

Et dépend des paramètres suivants

Paramètres	Entité	Description	Unité	Valeur
facteur_saison	Climat	Coefficient d'ajustement saisonnier	sans dimension	1.2

Indicateur PA.07.01 : Taux de renouvellement du troupeau (%/an)

Description

Proportion d'animaux remplacés annuellement. Indicateur de gestion démographique.

Calcul

```
TRT = (Nombre sorties) / (Effectif moyen) * 100
```

Attributs

L’indicateur dépend des variables suivantes :

variables	Entité	Description	Unité
nombre_sorties	Troupeau	Animaux vendus ou décédés	têtes
effectif_moyen	Troupeau	Effectif moyen annuel du troupeau	têtes

Et dépend des paramètres suivants

Paramètres	Entité	Description	Unité	Valeur
taux_optimal	Troupeau	Valeur cible pour une gestion durable	%	20

Indicateur PA.08.01 : Pression pastorale équivalente (UBT/ha/indice)

Description

Charge animale ajustée par la productivité des parcours. Permet des comparaisons intersites.

Calcul

```
PPE = (UBT) / (Indice productivité)
```

Attributs

L’indicateur dépend des variables suivantes :

variables	Entité	Description	Unité
unite_betail_total	Troupeau	Nombre d'unités de bétail tropical	UBT
indice_productivite	Parcours	Productivité fourragère du parcours	kg MS/ha

Et dépend des paramètres suivants

Paramètres	Entité	Description	Unité	Valeur
capacite_charge	Parcours	Charge animale soutenable à long terme	UBT/ha	0.5
## Agroécologie				

Indicateur AE.01.01 : Indice de diversité culturale (n.d.)

Description

Mesure la diversité des cultures selon l'indice de Shannon. Corrélé à la résilience du système.

Calcul

```
IDC = -Σ(p_i * ln(p_i))
```

Attributs

L'indicateur dépend des variables suivantes :

variables	Entité	Description	Unité
p_i	Parcelle	Proportion de la culture i dans la rotation	[0-1]

Et dépend des paramètres suivants

Paramètres	Entité	Description	Unité	Valeur
seuil_diversite	Système	Valeur cible pour une diversité optimale	indice	1.8

Indicateur AE.02.01 : Taux de couverture végétale (%)

Description

Pourcentage de sol couvert par la végétation vivante ou morte. Réduit l'érosion et améliore l'infiltration.

Calcul

```
TCV = (Surface couverte) / (Surface totale) * 100
```

Attributs

L'indicateur dépend des variables suivantes :

variables	Entité	Description	Unité
surface_couverte	Parcelle	Surface sous couvert végétal	ha
surface_totale	Parcelle	Surface totale de l'unité d'observation	ha

Et dépend des paramètres suivants

Paramètres	Entité	Description	Unité	Valeur
seuil_optimal	Système	Couverture végétale cible	%	70

Indicateur AE.03.01 : Indice de connectivité biologique (ha.pts)

Description

Mesure la connectivité des habitats naturels dans la matrice agricole. Essentiel pour la biodiversité fonctionnelle.

Calcul

```
ICB = Σ(Superficie corridor_i * Pondération_i)
```

Attributs

L'indicateur dépend des variables suivantes :

variables	Entité	Description	Unité
superficie_corridor	Paysage	Superficie du corridor écologique	ha
ponderation_ecologique	Paysage	Valeur de connectivité spécifique à l'habitat	points

Et dépend des paramètres suivants

Paramètres	Entité	Description	Unité	Valeur
seuil_connectivite	Paysage	Seuil minimum pour une connectivité fonctionnelle	sans dimension	0.75

Indicateur AE.04.01 : Taux de matière organique du sol (%)

Description

Concentration en matière organique dans les 20 premiers cm de sol. Proxy de la santé des sols.

Calcul

```
MOS = (Poids MO) / (Poids sol sec) * 100
```

Attributs

L'indicateur dépend des variables suivantes :

variables	Entité	Description	Unité
poids_matiere_organique	Sol	Poids sec de la matière organique	g
poids_sol_sec	Sol	Poids total du sol séché à l'étuve	g

Et dépend des paramètres suivants

Paramètres	Entité	Description	Unité	Valeur
profondeur_prelevement	Sol	Profondeur standard d'échantillonnage	cm	20

Indicateur AE.05.01 : Efficience d'utilisation de l'eau (kg/m³)

Description

Biomasse produite par unité d'eau consommée. Critique en zones arides.

Calcul

```
WUE = (Production biomasse) / (ETR)
```

Attributs

L'indicateur dépend des variables suivantes :

variables	Entité	Description	Unité
production_biomasse	Culture	Biomasse aérienne sèche produite	kg/ha
evapotranspiration	Climat	Evapotranspiration réelle de la culture	mm

Et dépend des paramètres suivants

Paramètres	Entité	Description	Unité	Valeur
coefficient_culture	Culture	Coefficient spécifique à l'espèce cultivée	sans dimension	0.85

Indicateur AE.06.01 : Indice d'autonomie en intrants (%)

Description

Part des intrants produits sur l'exploitation. Mesure la circularité du système.

Calcul

```
IAI = (Intrants internes) / (Intrants totaux) * 100
```

Attributs

L'indicateur dépend des variables suivantes :

variables	Entité	Description	Unité
intrants_internes	Système	Intrants produits sur l'exploitation	kg
intrants_totaux	Système	Consommation totale d'intrants	kg

Et dépend des paramètres suivants

Paramètres	Entité	Description	Unité	Valeur
seuil_autonomie	Système	Niveau minimal pour une exploitation durable	%	60

Indicateur AE.07.01 : Indice de spécialisation parcellaire (fonctions/parcelle)

Description

Nombre moyen de fonctions par parcelle (production, biodiversité...). Inverse de la spécialisation.

Calcul

```
ISP = Σ(Fonctions_i) / (Nombre parcelles)
```

Attributs

L'indicateur dépend des variables suivantes :

variables	Entité	Description	Unité
nombre_fonctions	Parcelle	Fonctions identifiées par parcelle	nombre
nombre_parcelles	Système	Parcelles totales dans l'exploitation	nombre

Et dépend des paramètres suivants

Paramètres	Entité	Description	Unité	Valeur
poids_fonction	Système	Coefficient d'importance relative des fonctions	sans dimension	1.0

Indicateur AE.07.02 : Diversité culturale (espèces/parcelle)

Description

Nombre moyen d'espèces cultivées par parcelle

Calcul

```
Σ(espèces_i) / (surface_observée)
```

Attributs

L'indicateur dépend des variables suivantes :

variables	Entité	Description	Unité
especes_cultivees	Parcelle	Nombre d'espèces distinctes cultivées	nombre

Et dépend des paramètres suivants

Paramètres	Entité	Description	Unité	Valeur
surface_observée	Espace	Surface échantillonnée	ha	1.0

Indicateur AE.07.03 : Diversité culturale (espèces/parcelle)

Description

Nombre moyen d'espèces cultivées par parcelle

Calcul

```
Σ(espèces_i) / (nombre_parcelles)
```

Attributs

L'indicateur dépend des variables suivantes :

variables	Entité	Description	Unité
especes_cultivees	Parcelle	Nombre d'espèces identifiées	nombre

Et dépend des paramètres suivants

Paramètres	Entité	Description	Unité	Valeur
poids_espece	Système	Coefficient de valeur écologique	sans dimension	1.2
## Environnemental				

Indicateur CA.01.01 : Tendance d'évolution de la pluviométrie (% de variation annuelle)

Description

Analyse l'évolution de la pluviométrie dans le territoire. Indicateur de caractérisation de la vulnérabilité climatique. Valeurs négatives = baisse, positives = hausse.

Calcul

```
CA.01.01 = a * CA.01.01 + (pluviométrie - pluviométrie-1)
```

Attributs

L'indicateur dépend des variables suivantes :

variables	Entité	Description	Unité
pluviométrie	Climat	La quantité de pluie annuelle cette année	mm/an
pluviométrie-1	Climat	La quantité de pluie annuelle l'année précédente (n-1)	mm/an

Et dépend des paramètres suivants

Paramètres	Entité	Description	Unité	Valeur
a	Climat	Facteur d'amortissement du passé	sans	0.1
## Démographique				

Indicateur CA.02.01 : Taux de croissance annuelle de la population (%)

Description

Croissance démographique dans les terroirs sur 30 ans. Indicateur de pression anthropique (2,6% aux derniers calculs).

Calcul

```
Donnée directe (pas de calcul)
```

Attributs

L'indicateur dépend des variables suivantes :

variables	Entité	Description	Unité
...	...	...	...

Et dépend des paramètres suivants

Paramètres	Entité	Description	Unité	Valeur
taux-croissance	ActeurIndividuel	Taux de croissance démographique du foyer	%	2.6
## Économique				

Indicateur EC.01.01 : Rentabilité moyenne des exploitations (€/ha/an)

Description

Marge brute moyenne par hectare cultivé

Calcul

```
(revenu_total - couts_production) / surface_totale
```

Attributs

L'indicateur dépend des variables suivantes :

variables	Entité	Description	Unité
revenu_total	Exploitation	Chiffre d'affaires annuel	€/an
couts_production	Exploitation	Coûts opérationnels annuels	€/an

Et dépend des paramètres suivants

Paramètres	Entité	Description	Unité	Valeur

Paramètres	Entité	Description	Unité	Valeur
surface_totale	Espace	Surface cultivée totale	ha	1.0
## Social				

Indicateur SO.01.01 : Taux d'emploi local (%)

Description

Part de la main d'œuvre issue du territoire

Calcul

```
emplois_locaux / emplois_totaux * 100
```

Attributs

L'indicateur dépend des variables suivantes :

variables	Entité	Description	Unité
emplois_locaux	ActeurIndividuel	Nombre de travailleurs résidant dans le territoire	personnes
emplois_totaux	ActeurIndividuel	Nombre total de travailleurs	personnes

Et dépend des paramètres suivants

Paramètres	Entité	Description	Unité	Valeur
...	...	...	...	...
## Climat				

Indicateur CL.03.01 : Bilan carbone (tCO2eq/ha/an)

Description

Émissions nettes de GES par hectare

Calcul

```
(émissions - séquestration) / surface
```

Attributs

L'indicateur dépend des variables suivantes :

variables	Entité	Description	Unité

variables	Entité	Description	Unité
émissions	Exploitation	Émissions totales de GES	tCO2eq/an
séquestration	Sol	Carbone séquestré dans les sols	tCO2eq/an

Et dépend des paramètres suivants

Paramètres	Entité	Description	Unité	Valeur
surface	Espace	Surface agricole utile	ha	1.0

Indicateur CL.03.02 : Indice de résilience climatique (%)

Description

Capacité à maintenir la production sous stress climatique

Calcul

```
(production_reelle / production_potentielle) * 100
```

Attributs

L'indicateur dépend des variables suivantes :

variables	Entité	Description	Unité
production_reelle	Exploitation	Production effective en année stressante	t/ha
production_potentielle	Exploitation	Production théorique optimale	t/ha

Et dépend des paramètres suivants

Paramètres	Entité	Description	Unité	Valeur
...	...	...	...	...
## Biodiversité				

Indicateur BD.04.01 : Indice de biodiversité fonctionnelle (points)

Description

Abondance relative d'espèces indicatrices

Calcul

```
Σ(abondance_espèce_i * poids_écologique_i)
```

Attributs

L'indicateur dépend des variables suivantes :

variables	Entité	Description	Unité
abondance_especes	Écosystème	Nombre d'individus par espèce	individus/ha

Et dépend des paramètres suivants

Paramètres	Entité	Description	Unité	Valeur
poids_ecologique	Système	Valeur écologique de l'espèce	coefficient	0.75

Indicateur BD.04.02 : Indice de biodiversité (espèces/ha)

Description

Richesse spécifique normalisée

Calcul

```
nombre_espèces / surface_reference
```

Attributs

L'indicateur dépend des variables suivantes :

variables	Entité	Description	Unité
nombre_espèces	Espace	Espèces identifiées	nombre

Et dépend des paramètres suivants

Paramètres	Entité	Description	Unité	Valeur
surface_reference	Système	Surface de référence	ha	10.0

Initialisation

Initialisation

Portée Temporelle

Configuration de la dimension temporelle de la simulation :

Paramètre	Valeur	Description
Date de début	`2020-01-01`	Date initiale de la simulation
Date de fin	`2030-12-31`	Date finale de la simulation
Pas de temps	`1 day`	Incrément temporel de simulation
Période de chauffe	`365 days`	Période d'initialisation avant collecte des données

Configuration Spatiale

Configuration de la dimension spatiale de la simulation :

Paramètre	Valeur	Description
Extension	`[min_x, min_y, max_x, max_y]`	Bounding box de la zone d'étude
Projection	`EPSG:4326`	Système de coordonnées de référence
Résolution	`100 m`	Résolution spatiale des données

Données d'Entrée

Documentation

Description des différentes sources de données utilisées pour l'initialisation du modèle.

Cette section recense l'ensemble des données d'entrée nécessaires au fonctionnement du modèle, incluant les données spatiales, tabulaires et autres sources de données.

Données Spatiales

Type	Format	Chemin	Description	Résolution	Qualité
raster	GeoTIFF	`data/land_use.tif`	CORINE Land Cover 2020	100m	low
vector	GeoJSON	`data/parcels.geojson`	Agricultural parcel boundaries	...	high

Données Tabulaires

Type	Format	Chemin	Description	Taille	Taux Réponse	Validation
csv	...	`data/household_survey.csv`	Socio-economic survey 2020	1500	78%%	cross_validation

Entités

Documentation

Description des entités, agents et objets du modèle.

Cette section spécifie comment chaque entité est initialisée et quelles données sont utilisées pour leur création.

Liste des Entités

Nom	Type	Effectif	Méthode d'Initialisation
HouseholdAgent	agent	1500	data_driven
ParcelEntity	environment	N/A	gis_import

Détails par Entité

Entité : HouseholdAgent

Type : agent

Effectif : 1500

Méthode d'initialisation : `data_driven`

Source de données : `data/household_survey.csv`

Stratégie d'échantillonnage : `stratified_random`

Variables de stratification : `income_class`, `location`

Entité : ParcelEntity

Type : environment

Méthode d'initialisation : `gis_import`

Source de données : `data/parcels.geojson`

Mapping d'attributs : - `size_ha` ← `area` (conversion: `square_meters_to_hectares`)

Paramètres

Documentation

Cette section présente l'ensemble des paramètres du modèle, divisés en deux catégories :

- **Paramètres fixes (Z)** : Valeurs constantes pour toutes les simulations
- **Paramètres variables (X)** : Valeurs à explorer dans les scénarios

Paramètres Fixes (Z)

Paramètres dont la valeur est constante pour toutes les simulations :

Paramètre	Entité	Définition	Unité	Valeur	Incertitude	Sensibilité
`discount_rate`	HouseholdAgent	Temporal discount rate for decision-making	dimensionless	**0.03**	Dist: normal, σ: 0.005	high
`soil_mineralization_rate`	ParcelEntity	Rate of organic matter mineralization	kgN/kgOM/day	**0.15**	Non quantifié	...

Paramètres Variables (X)

Paramètres à explorer dans les scénarios :

Paramètre	Entité	Définition	Unité	Valeurs	Impact
`climate_change_scenario`	Global	IPCC climate scenarios	...	**RCP2.6, RCP4.5, RCP8.5**	high
`policy_intervention`	Global	Policy intervention scenarios	...	**baseline, subsidy, tax, regulation**	...

Variables d'État Initiales

Documentation

Les variables d'état représentent l'état initial du système au début de la simulation.

Variables fixes : Valeurs initiales constantes pour toutes les simulations
Variables variables : Plages de valeurs à explorer dans les scénarios

Variables d'État Fixes

Valeurs initiales constantes pour toutes les simulations :

Variable	Entité	Valeur	Unité	Distribution	Paramètres
`initial_wealth`	HouseholdAgent	**50000**	EUR	lognormal	mean=10.82, sigma=0.8

Variables d'État Variables

Plages de valeurs initiales à explorer dans les scénarios :

Variable	Entité	Valeurs	Unité	Distribution	Probabilités
`land_use_type`	ParcelEntity	forest, agriculture, ..., wetland	...	categorical	0.30, 0.40, ..., 0.10

Scénarios

Documentation

Description des scénarios que l'on veut explorer avec justifications et références éventuelles.

Un scénario est défini par l'attribution de valeurs à un sous-ensemble X des paramètres et variables énumérées dans la section Initialisation.

Détails par Scénario

Scénario : sustainability_transition

Description : Pathway towards sustainable agroecology

Base : IPCC RCP2.6 + EU Green Deal

Statut de validation : plausible

Paramètres : - `climate_change_scenario` = `RCP2.6` - `policy_intervention` = `regulation`

Variables d'état initiales : - `initial_investment` = `20000`

Scénario : business_as_usual

Description : Current trends continuation

Base : Non spécifié

Statut de validation : calibrated

Paramètres : - `climate_change_scenario` = `RCP4.5` - `policy_intervention` = `baseline`

Variables d'état initiales : - `initial_investment` = `10000`

Règles d'Initialisation

Documentation

Description des règles et méthodes utilisées pour initialiser les entités du modèle.

Les règles d'initialisation définissent comment les valeurs initiales sont attribuées aux différentes entités et variables du modèle, incluant les distributions statistiques, les méthodes de calcul et les hypothèses sous-jacentes.

Liste des Règles d'Initialisation

Entité	Règle	Implémentation	Hypothèses
HouseholdAgent	`wealth ~ lognormal(10.82, 0.8)`	`numpy.random.lognormal(10.82, 0.8, n_agents)`	`Wealth distribution follows national statistics`

Entité	Règle	Implémentation	Hypothèses
ParcelEntity	`land_use = categorical([0.3, 0.4, 0.2, 0.1])`	`random.choice(['forest', 'agriculture', 'urban', 'wetland'], p=[0.3, 0.4, 0.2, 0.1])`	`...`

Détails par Règle

Règle 1 : HouseholdAgent

Règle mathématique :

```
wealth ~ lognormal(10.82, 0.8)
```

Implémentation code :

```
numpy.random.lognormal(10.82, 0.8, n_agents)
```

Hypothèses et justifications : Wealth distribution follows national statistics

Règle 2 : ParcelEntity

Règle mathématique :

```
land_use = categorical([0.3, 0.4, 0.2, 0.1])
```

Implémentation code :

```
random.choice(['forest', 'agriculture', 'urban', 'wetland'], p=[0.3, 0.4, 0.2, 0.1])
```

Hypothèses : *Aucune hypothèse spécifiée*

Mode d'emploi

Lancement de la simulation

Configuration requise

Système d'exploitation	Windows 10/11, Linux Ubuntu 20.04+, macOS 12+
Mémoire RAM	16GB recommandés (8GB minimum)
Espace disque	2GB libre (données spatiales incluses)

Dépendances	GAMA Platform 1.9.1+, Java 17+, GDAL 3.4+ (pour données géospatiales), Python 3.10+ (pour analyse post-simulation)

Fichiers nécessaires

Fichier	Description	Format	Exemple
`main_model.gaml`	Modèle principal GAMA contenant les espèces et expériences	GAML	`models/pastoral_abm/main_model.gaml`
`config_parameters.xml`	Fichier de paramétrage de la simulation	XML	`N/A`
`land_cover.tiff`	Carte d'occupation du sol (classification FAO LCCS)	GeoTIFF	`N/A`
`herd_data.csv`	Données initiales des troupeaux par ménage	CSV UTF-8	`N/A`

Procédure de lancement

Étape 1 : Lancement en mode headless (sans interface)

```
gama-headless.exe main_model.gaml config_parameters.xml
```

Options disponibles : - --experiment main_experiment : exécute l'expérience principale - --output results/ : spécifie le dossier de sortie - --params param1=value1 param2=value2 : paramètres en ligne de commande

Étape 2 : Lancement avec interface graphique GAMA

```
gama.exe main_model.gaml
```

Prérequis : Environnement graphique disponible

Étape 3 : Lancement batch de multiples scénarios

```
python run_batch.py --scenarios all --replicates 10
```

Prérequis : Script Python d'orchestration installé

Les entrées

Interface utilisateur

Type	GAMA Modeling Environment

URL	`https://gama-platform.org`
Fonctionnalités	Exploration visuelle des scénarios climatiques (X), Modification interactive des paramètres (Z) : taux de reproduction, règles de mobilité, Visualisation temps réel de la dynamique pastorale, Inspection des agents (troupeaux, ménages, ressources)

Fichiers d'entrée

Type	Description	Format	Détails
parametres_globaux	Paramètres globaux du modèle socio-écologique	XML	Champs obligatoires: simulation_duration: 3650 (10 ans), time_step: 1 (jour), climate_scenario: RCP4.5
donnees_spatiales	*Information à compléter*	GeoTIFF/Shapefile	Système: WGS84/UTM
donnees_socio_economiques	Données des ménages et troupeaux	CSV/JSON	Aucun
donnees_climatiques	Données climatiques historiques et projections	NetCDF	Aucun

Les sorties

Visualisation

Type	Description	Format	Outils
cartes_dynamiques	Cartographie dynamique de la mobilité pastorale et état des ressources	HTML interactif	GAMA GIS + Leaflet.js
courbes_evolution	Graphiques temporels des indicateurs clés	SVG/PDF interactif	GAMA Charts + Plotly
reseaux_sociaux	Visualisation des réseaux d'entraide et partage de ressources	GEXF/JSON	Gephy/Cytoscape

Fichiers de sortie

Type	Description	Format	Contenu

Type	Description	Format	Contenu
resultats_bruts	Données brutes pour analyse statistique	CSV	Colonnes: step: Pas de temps, household_id: Identifiant ménage, location_x, location_y: Coordonnées UTM, livestock_count: Effectif bétail (TLU), income: Revenu journalier, biomass_available: Biomasse disponible locale
metrics_agregees	Indicateurs agrégés par scénario	JSON	Données structurées
rapport_synthese	Rapport automatique avec graphiques et analyses	PDF/HTML	Sections: résumé_exécutif, performance_écologique, résilience_sociale, recommandations_gestions

Post-traitement

Outils recommandés : - R Studio (analyse statistique) - Jupyter Notebook (analyse exploratoire) - QGIS (analyse spatiale) - Gephi (analyse de réseaux)

Scripts d'exemple : `scripts/analysis/`
